

리눅스 기초

inky4832@daum.net

1.1 리눅스 기초

리눅스 (Linux)

- 핀란드 헬싱키대학교의 학생이었던 리누스 베네딕트 토발즈(Linus Benedict Torvals)가 처음 개발
- 미닉스(MINIX)라는 교육용 운영체제를 참조하여 개발
- 리눅스 개발 소식을 comp.os.minix 뉴스 그룹에 포스팅: 1991년 8월 26일 -> 리눅스 탄생일

Hello everybody out there using minix-
I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones. This has been brewing since april, and is starting to get ready. I'd like any feedback on things people like/dislike in minix, as my OS resembles it somewhat (same physical layout of the file-system (due to practical reasons) among other things).

I've currently ported bash(1.08) and gcc(1.40), and things seem to work. This implies that I'll get something practical within a few months, and I'd like to know what features most people would want. Any suggestions are welcome, but I won't promise I'll implement them :-)

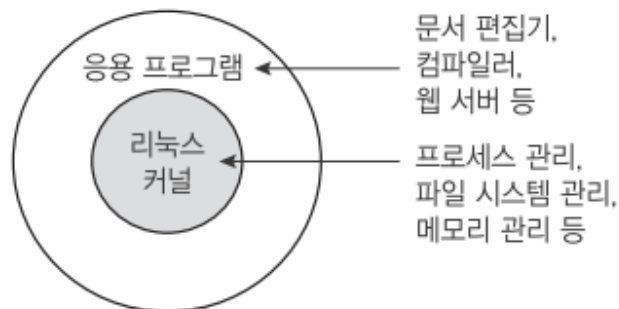
Linus (torv...@kruuna.helsinki.fi)

PS. Yes-it's free of any minix code, and it has a multi-threaded fs. It is NOT protable (uses 386 task switching etc), and it probably never will support anything other than AT-harddisks, as that's all I have :-).

1.1 리눅스 기초

발전과정

- 최초 공개된 리눅스 커널: 버전 0.01
- 현재(2025년) 안정버전 Ubuntu 24.04.1 LTS, Debian 13
- GNU 프로젝트: 리눅스 커널에 응용 프로그램 제공 -> GNU/리눅스
- 리눅스 재단: 2007년 설립
 - 리눅스 토발즈 지원
 - 삼성전자, IBM, 인텔, 오라클, 구글, 페이스북, 트위터 등
 - 2005년 이래 7,800명이 넘는 개인과 800여 개의 기업이 커널 개발에 공헌



1.1 리눅스 기초

GNU 프로젝트

- 리처드 스톨만이 시작
- 1985년 <GNU 선언문> 발표 및 자유소프트웨어재단(Free Software Foundation, FSF)'을 설립
 - <GNU 선언문>은 <http://www.gnu.org/gnu/manifesto.html>에서 확인
- GNU는 유닉스와 호환되는 자유 소프트웨어를 개발하는 프로젝트
 - 'GNU is Not Unix (GNU는 유닉스가 아니다)'의 약자로 '그누'라고 읽음.
- GNU는 다음과 같은 네 가지 자유를 보장(www.gnu.org).
 - 프로그램을 어떠한 목적으로도 실행할 수 있는 자유.
 - 프로그램이 어떻게 동작하는지 학습하고, 자신의 필요에 맞게 개작할 수 있는 자유. 이를 위해서는 소스 코드에 대한 접근이 전제되어야 한다.
 - 이웃을 도울 수 있도록 복제물을 재배포할 수 있는 자유.
 - 프로그램을 개선할 수 있는 자유와 개선된 이점을 공동체 전체가 누릴 수 있도록 발표할 자유. 이를 위해서도 역시 소스 코드에 대한 접근이 전제되어야 한다.
- 1989년에 GPL(GNU General Public License) 제정: 자유 소프트웨어 라이선스
 - 버전 1(GPLv1), 버전 2(GPLv2), 버전 3(GPLv3)
 - 컴퓨터 프로그램의 자유로운 사용, 무료 배포, 소스코드 변경 허용 등
 - www.gnu.org/licenses/licenses.html 참조

1.1 리눅스 기초

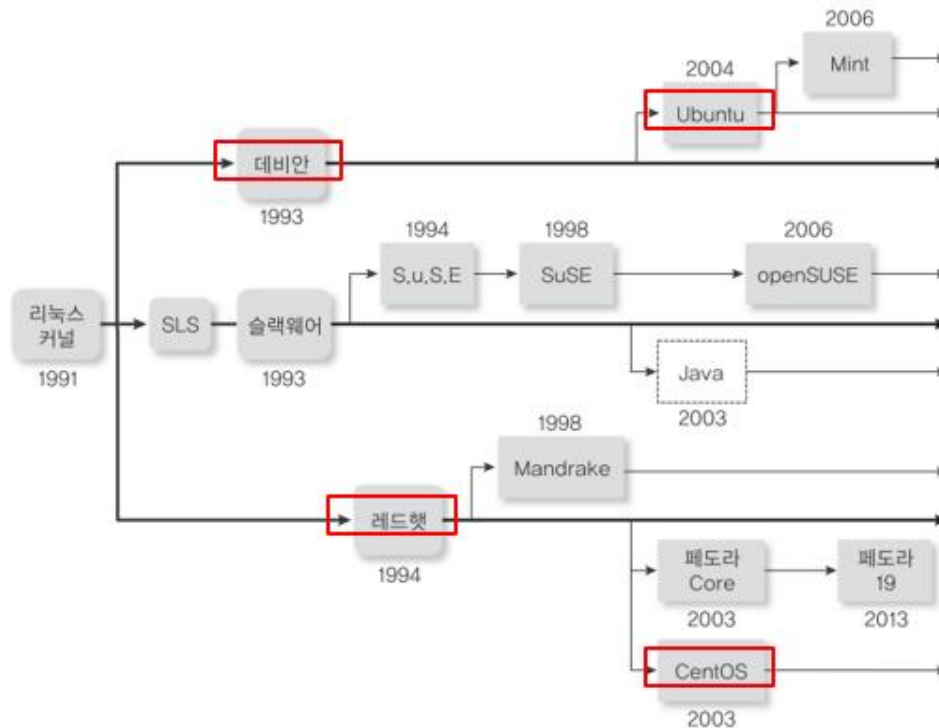
리눅스와 유닉스

- 리눅스는 유닉스 계열의 운영체제
- 유닉스
 - 1969년 AT&T의 벨연구소에서 어셈블리어로 처음 개발
 - 1971년에 C언어로 재개발 -> 최초의 고급 프로그래밍 언어로 작성한 운영체제로 이식성 높음
 - AT&T의 상용 유닉스와 오픈소스 버전인 BSD로 나뉘어 발전
 - BSD는 AT&T의 라이선스가 필요없는 FreeBSD로 발전

1.1 리눅스 기초

리눅스 배포판

- 리눅스 커널 + 응용프로그램으로 구성
- 레드햇 계열, 데비안 계열, 슬랙웨어 계열
- 리눅스 배포판 계통도: <https://namu.wiki/w/Linux/배포판>



Debian 기반: Ubuntu, Kali Linux
설치와 설정이 매우 쉽고 다양한 S/W를 매우 쉽게 사용 가능. 사용자 친화성에 더 중점을 둬. 파일 확장자가 .dev 로 끝남.

RPM 기반: RHEL, CentOS, Oracle Linux, Amazon Linux, Rocky Linux
매우 안정적이며 안전한 OS임. S/W는 레드햇 저장소에서만 다운로드 가능. 해당 S/W에 보안 또는 성능결함 여부를 매우 철처하게 테스트함. 파일 확장자가 .rpm 로 끝남

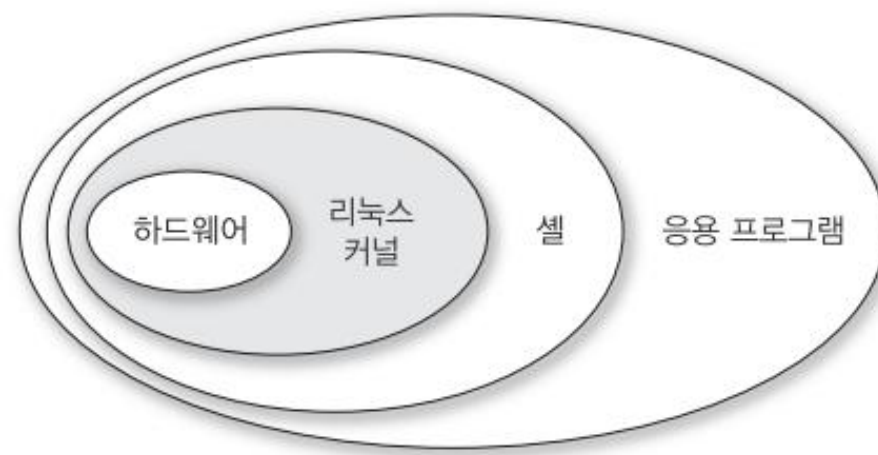
1.1 리눅스 기초

리눅스 특징

- 리눅스는 공개 소프트웨어이며 무료로 사용할 수 있다.
- 유닉스와의 완벽한 호환성을 유지한다.
- 서버용 운영체제로 많이 사용된다.
- 편리한 GUI 환경을 제공한다.

리눅스 구조

- 커널: 리눅스의 핵심
 - 프로세스/메모리/파일시스템/장치 관리
 - 컴퓨터의 모든 자원 초기화 및 제어 기능
- 셸(shell): 사용자 인터페이스
 - 명령해석
 - 프로그래밍기능
 - 리눅스 기본 셸: bash 셸(리눅스 셸)
- 응용 프로그램
 - 각종 프로그래밍 개발도구
 - 문서편집도구
 - 네트워크 관련 도구 등



1.2 실습 환경 구성

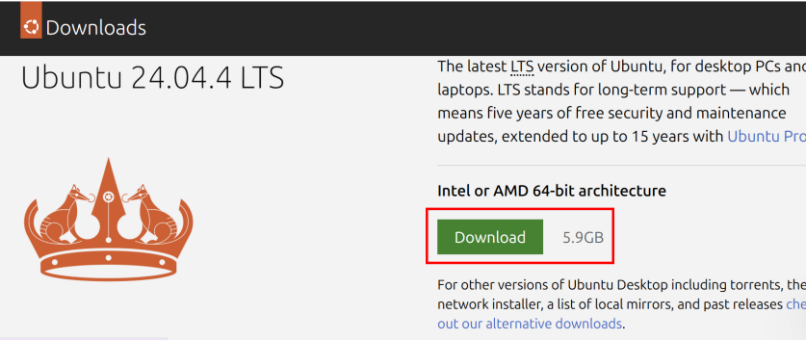
가상 머신

- PC에 설치되어 있는 운영체제(**호스트 os**)에 가상의 머신(시스템)을 생성한 후 여기에 다른 운영체제(**게스트 os**)를 설치할 수 있도록 해주는 응용 프로그램

가상머신	호스트os	게스트os
VMWare	윈도 계열, 대부분의 리눅스, 맥os	윈도 계열os, 대부분의 리눅스, 솔라리스, 맥os
VirtualPC	윈도 계열 OS	윈도 계열os, 일부 리눅스, 솔라리스
VirtualBox	윈도 계열, 대부분의 리눅스, 맥os, 솔라리스	윈도 계열os, 대부분의 리눅스, 솔라리스, 맥os, OpenBSD

ISO 이미지 다운로드 (ubuntu-24.04.4-desktop-amd64.iso)

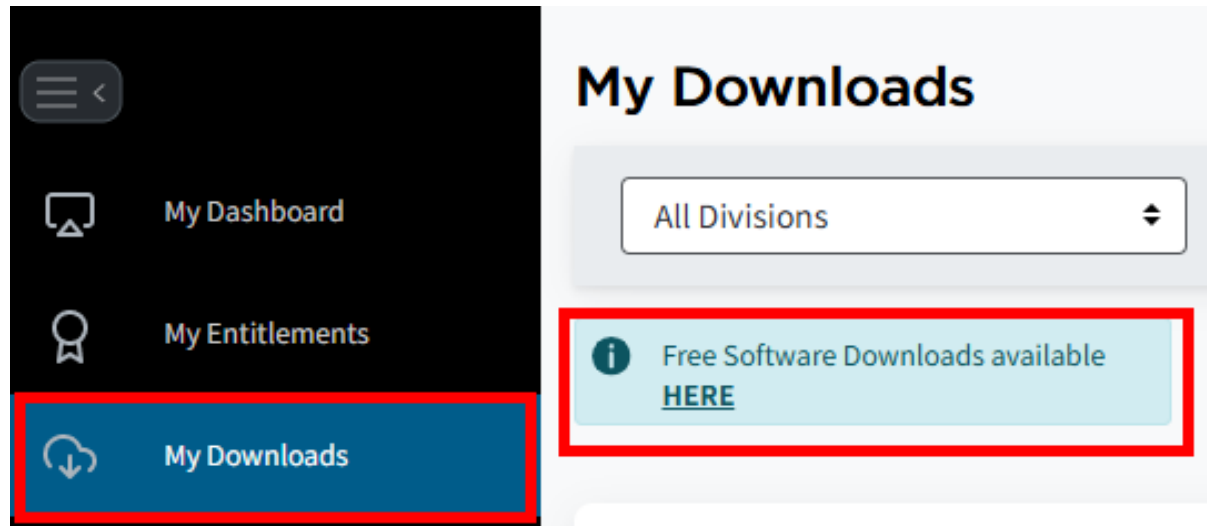
- <https://ubuntu.com/download/desktop>



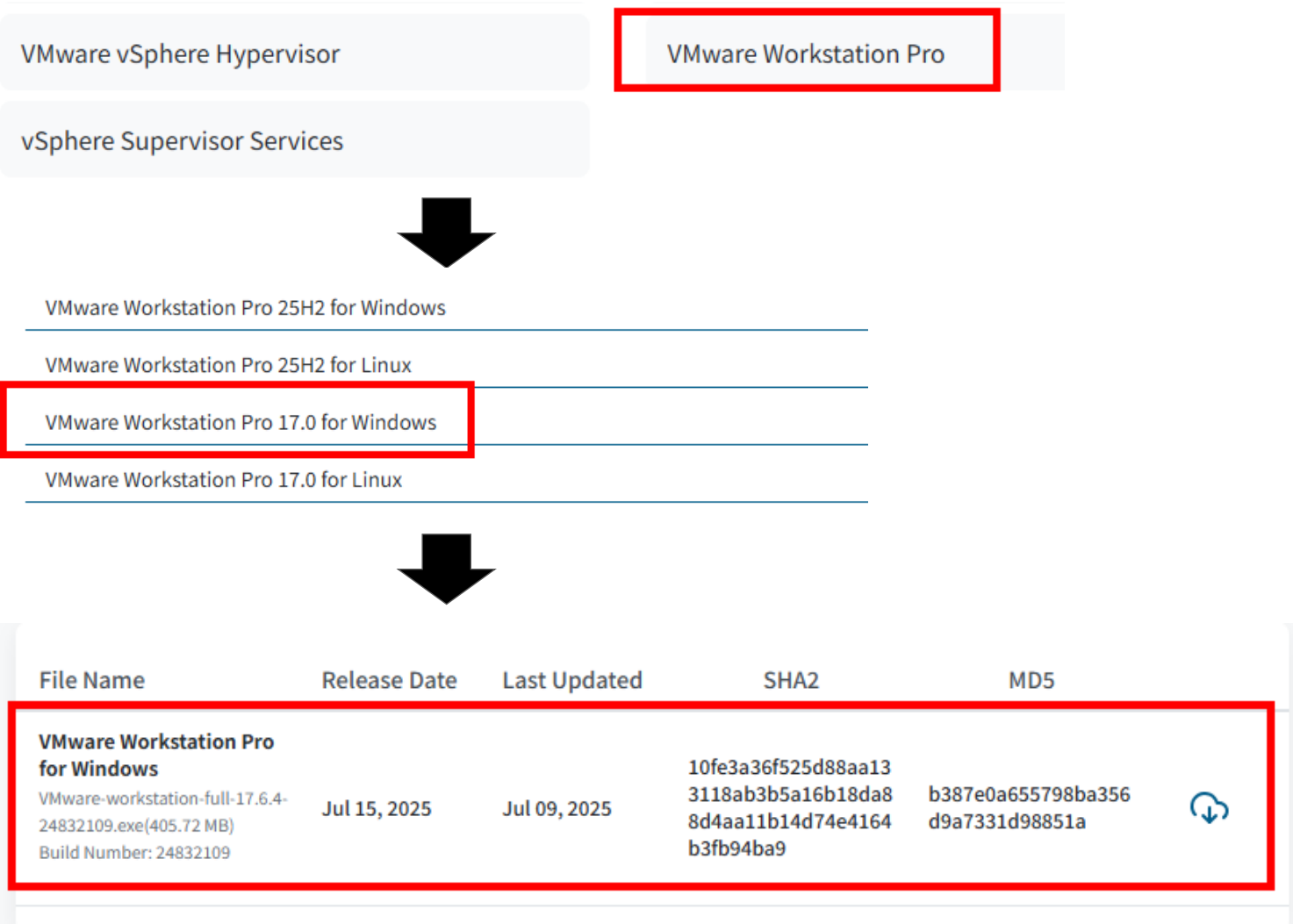
1.2 실습 환경 구성

VMWare 설치

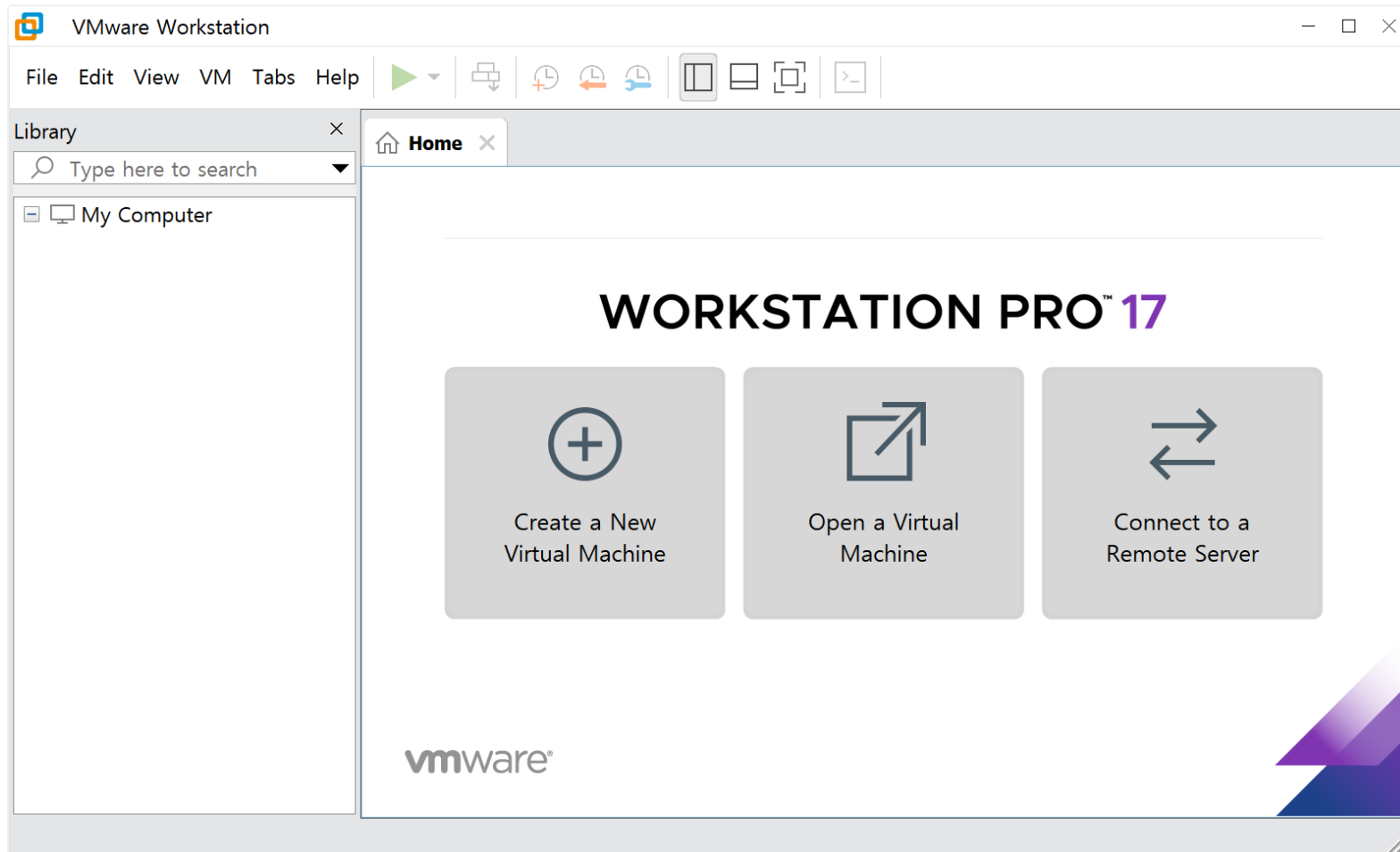
- <https://profile.broadcom.com/web/registration>
- <https://support.broadcom.com/group/ecx/free-downloads>



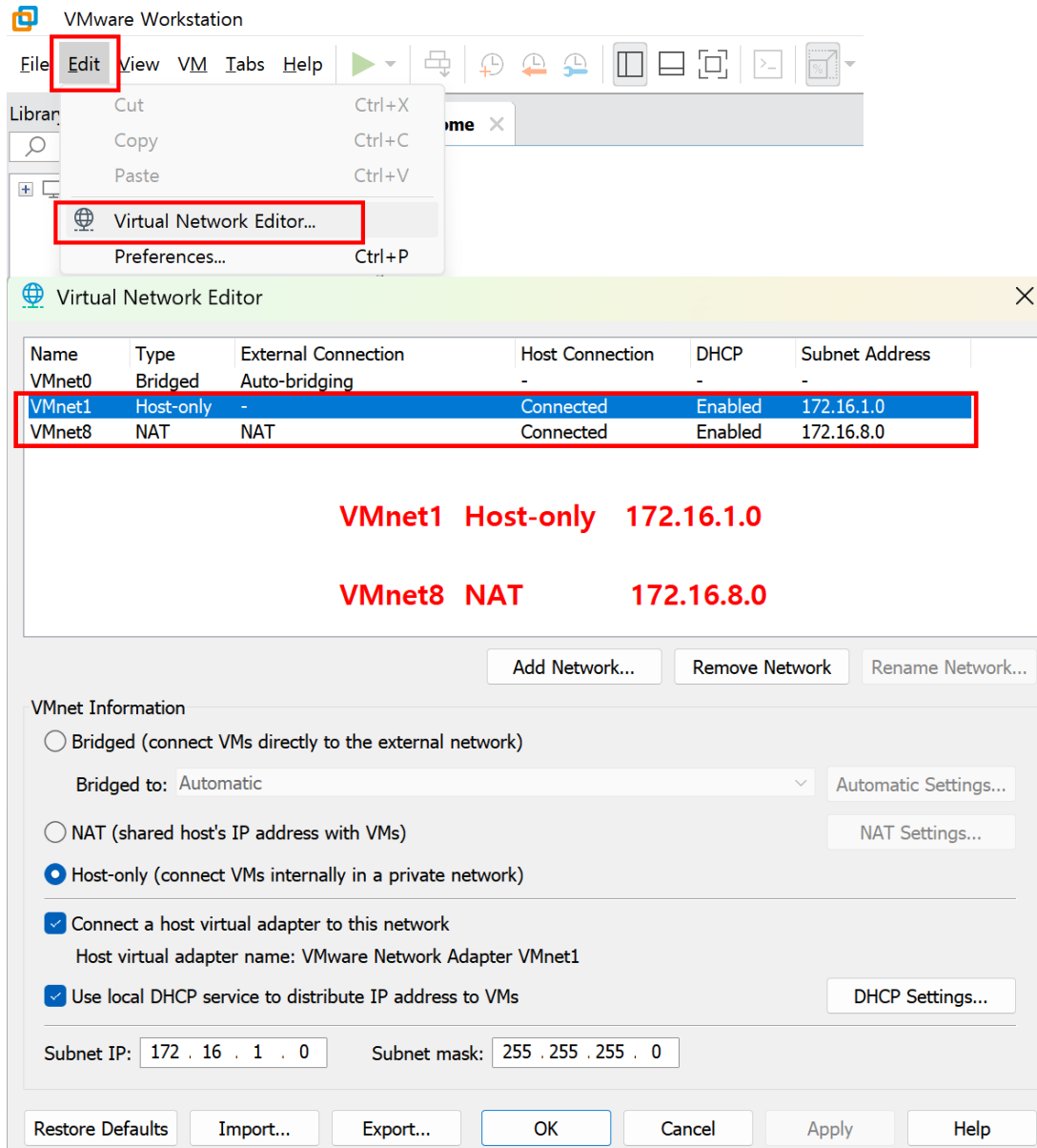
1.2 실습 환경 구성



1.2 실습 환경 구성



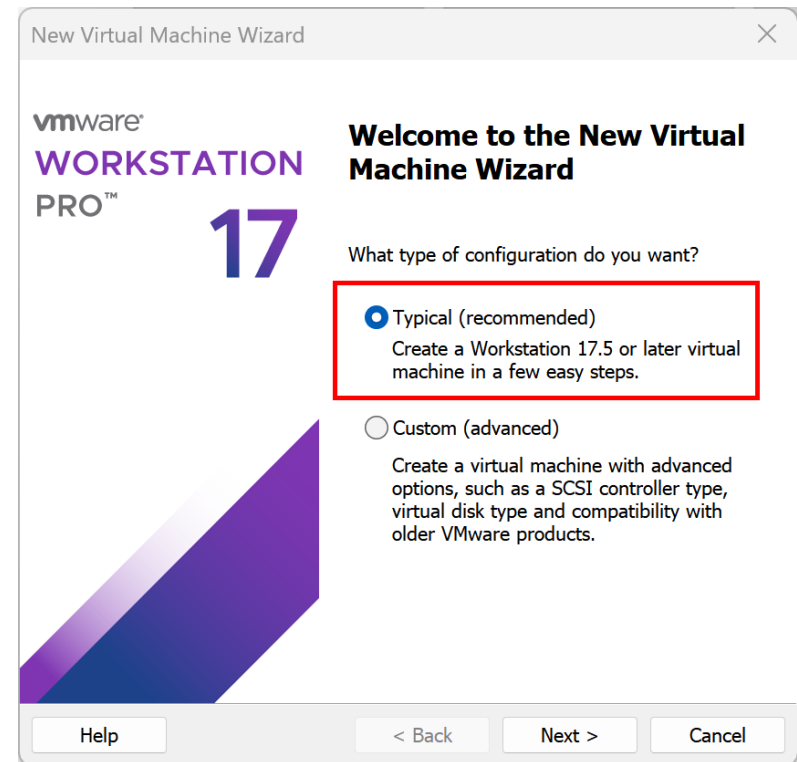
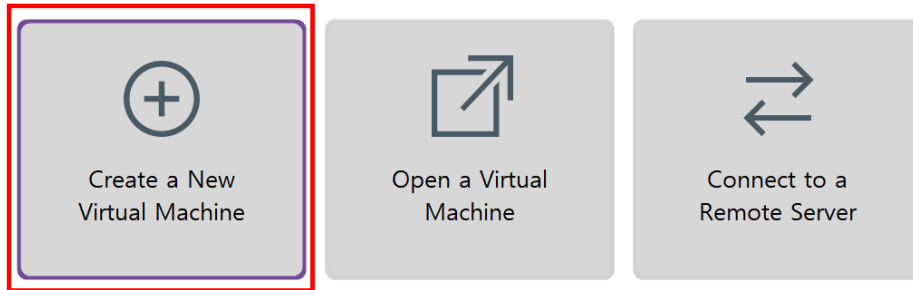
1.2 실습 환경 구성



1.2 실습 환경 구성

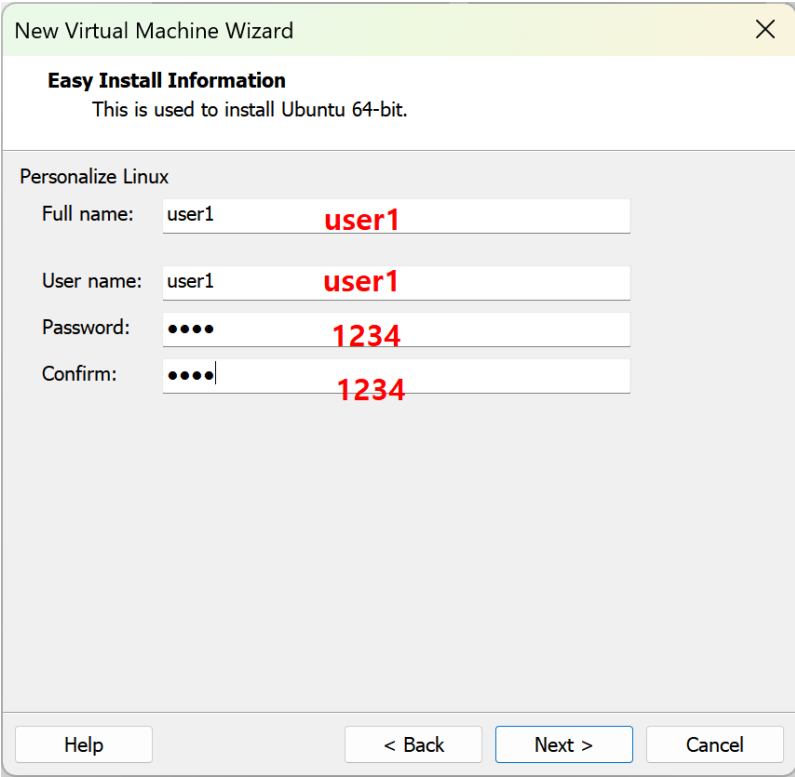
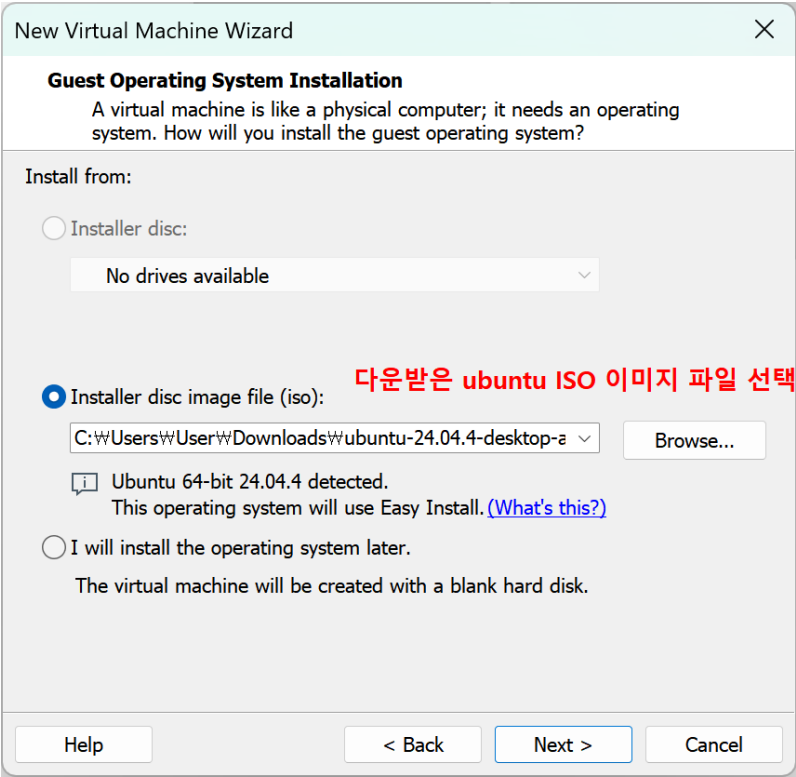
VMWare 에 Ubuntu 설치

WORKSTATION PRO™ 17



1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



1.2 실습 환경 구성

VMWare 에 Ubuntu 설치

New Virtual Machine Wizard

Name the Virtual Machine
What name would you like to use for this virtual machine?

Virtual machine name:
Ubuntu 64-bit

Location: **c:\vmware_ubuntu64**
C:\vmware_ubuntu64 Browse...

The default location can be changed at Edit > Preferences.

< Back Next > Cancel



New Virtual Machine Wizard

Specify Disk Capacity
How large do you want this disk to be?

The virtual machine's hard disk is stored as one or more files on the host computer's physical disk. These file(s) start small and become larger as you add applications, files, and data to your virtual machine.

Maximum disk size (GB): 20.0

Recommended size for Ubuntu 64-bit: 20 GB

☐ Store virtual disk as a single file

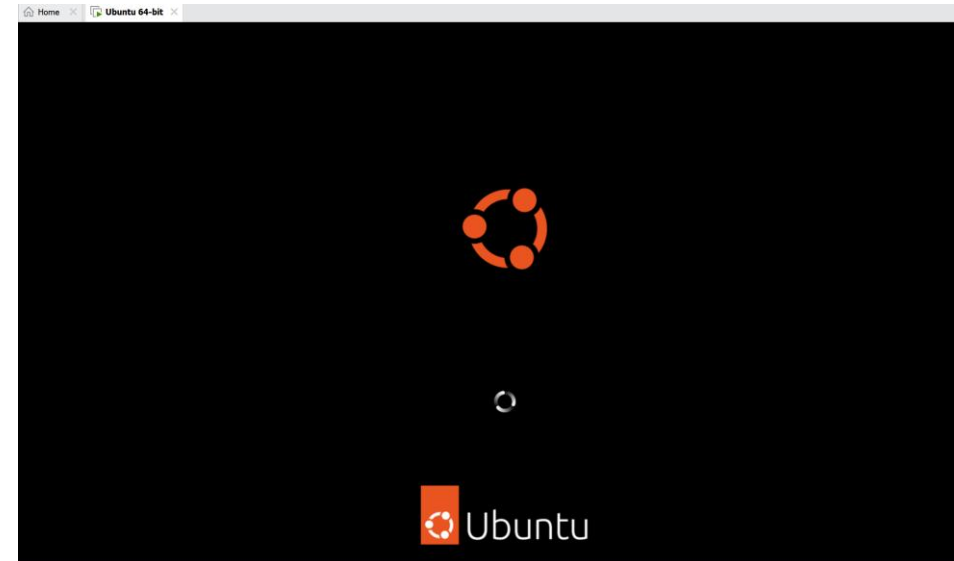
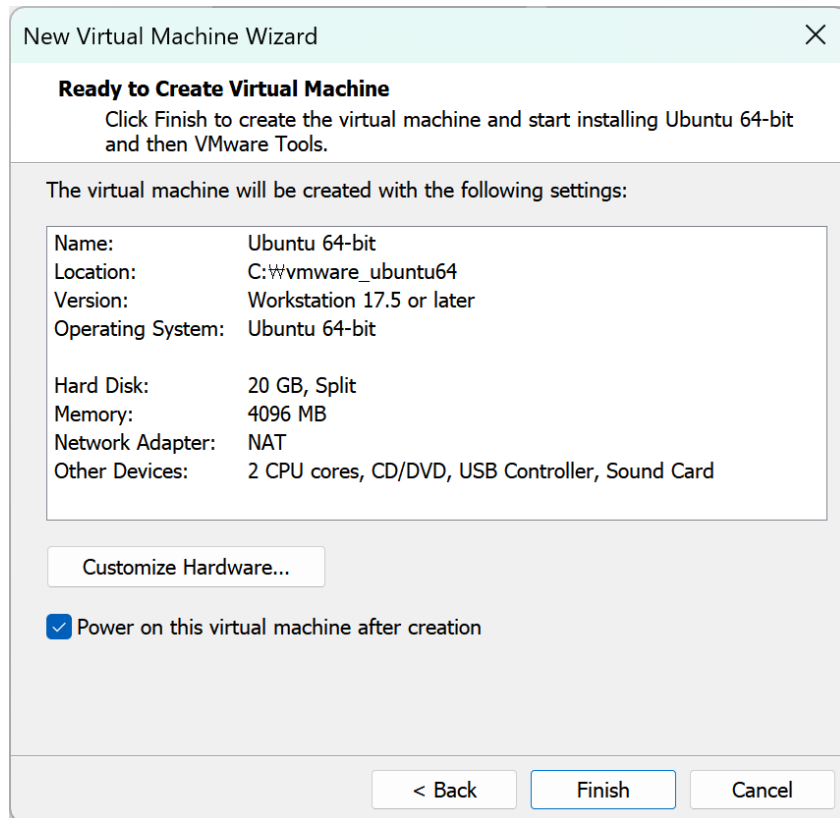
☒ Split virtual disk into multiple files

Splitting the disk makes it easier to move the virtual machine to another computer but may reduce performance with very large disks.

Help < Back Next > Cancel

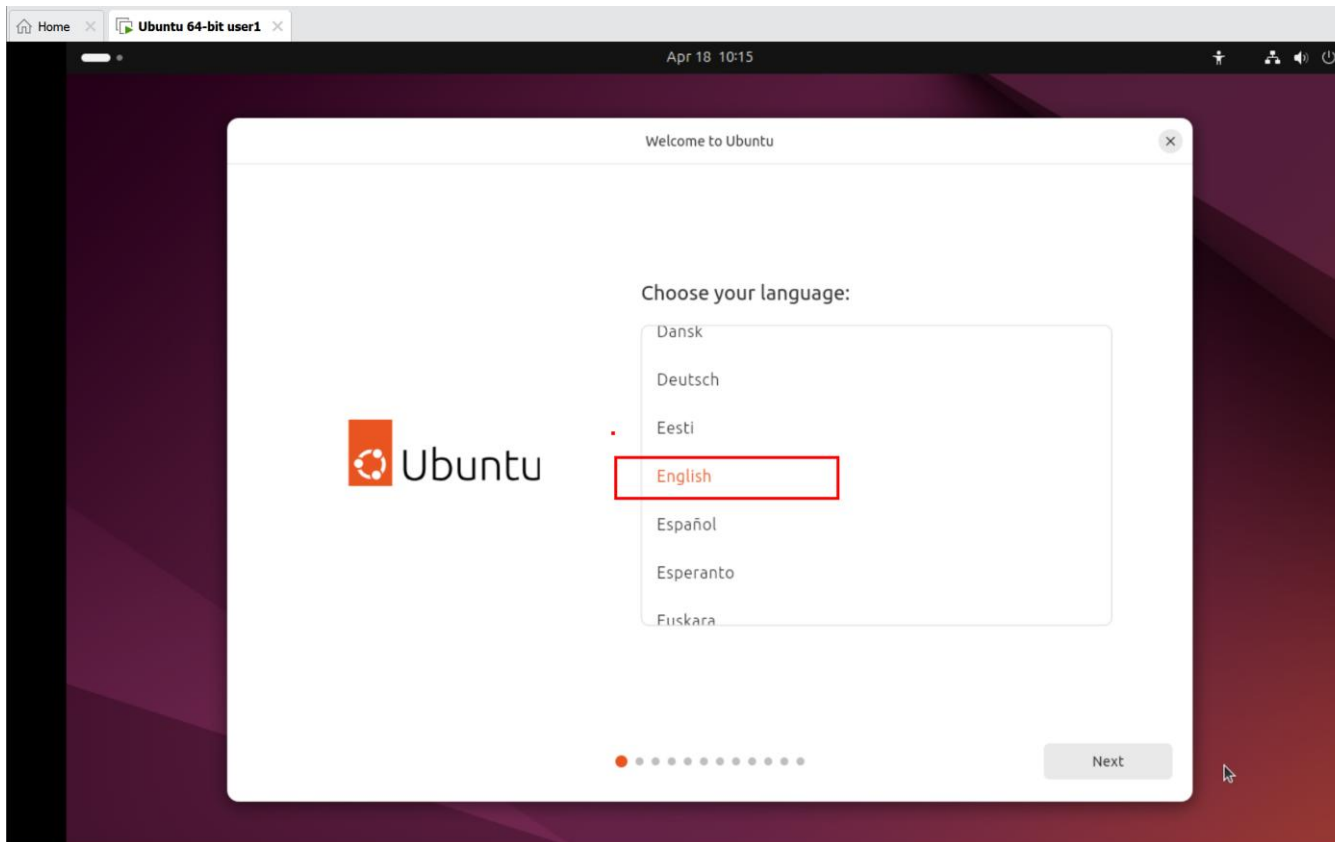
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



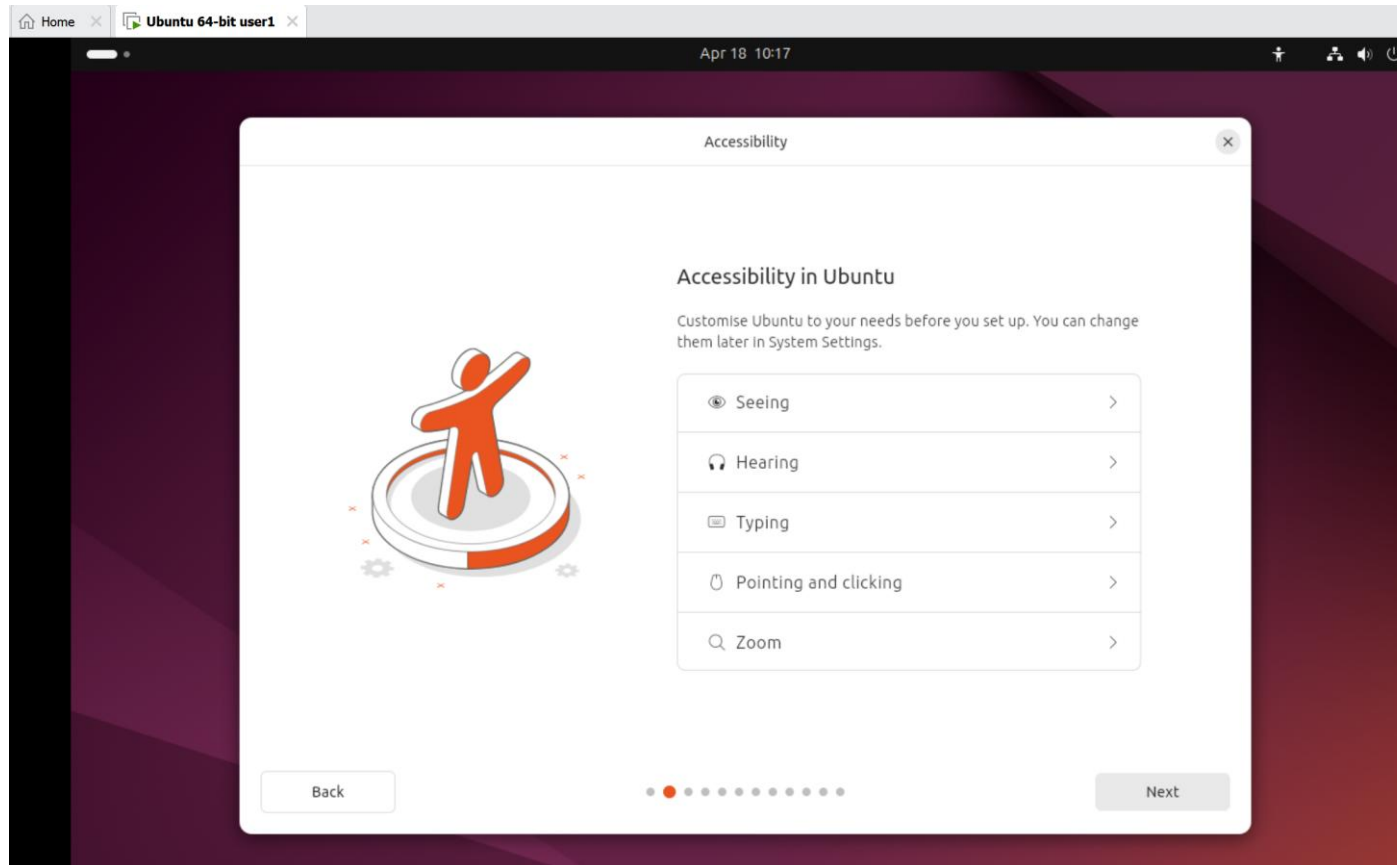
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



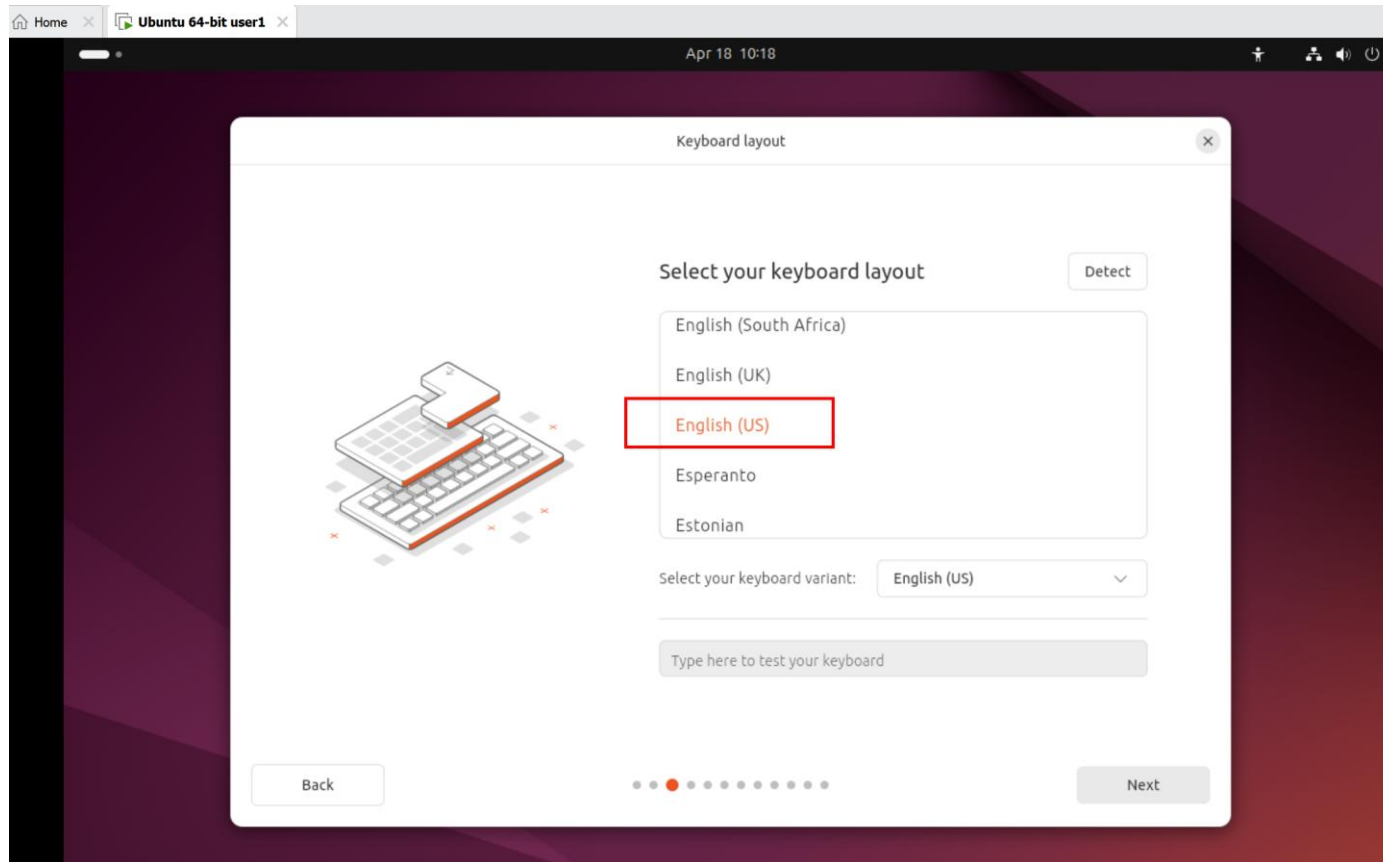
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



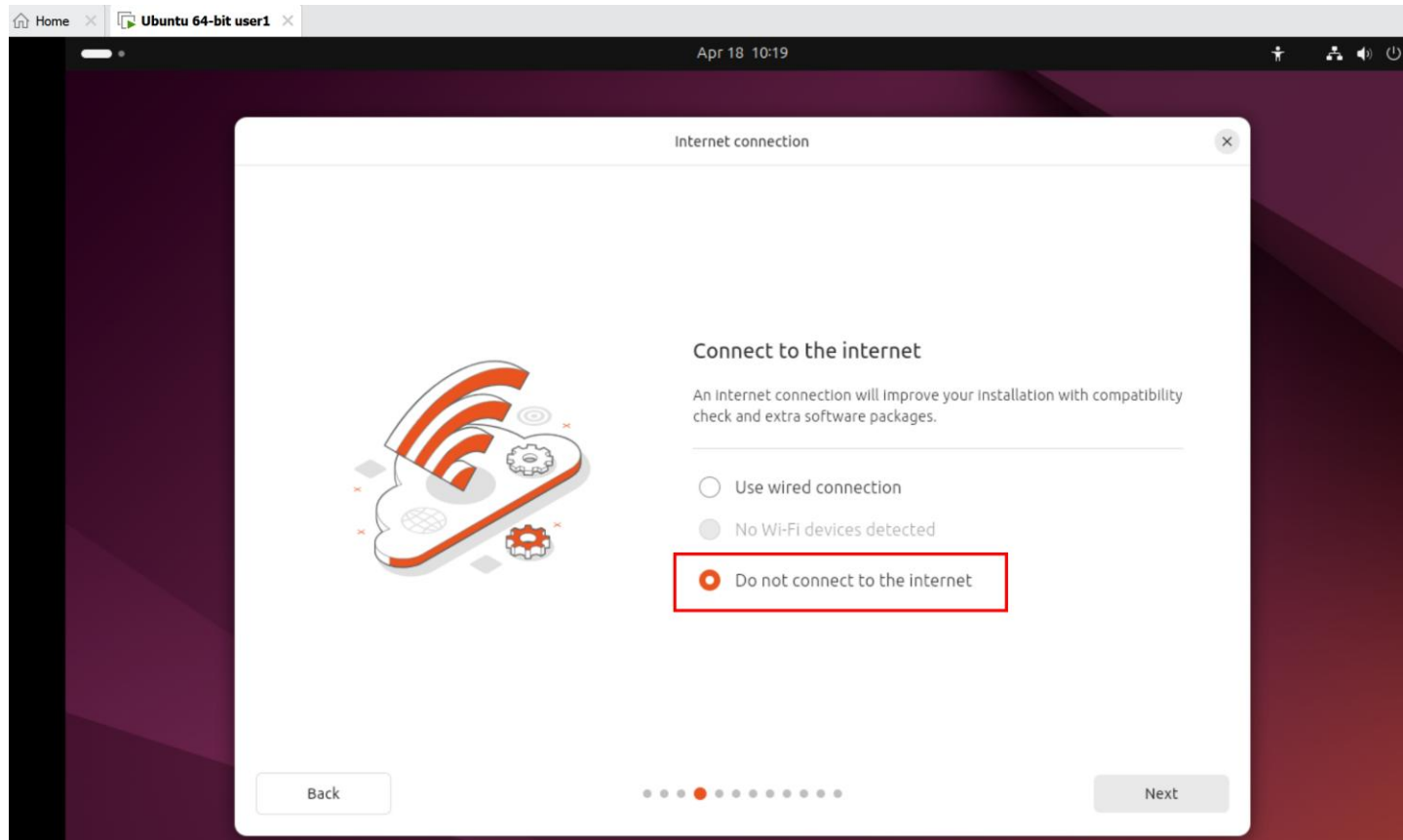
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



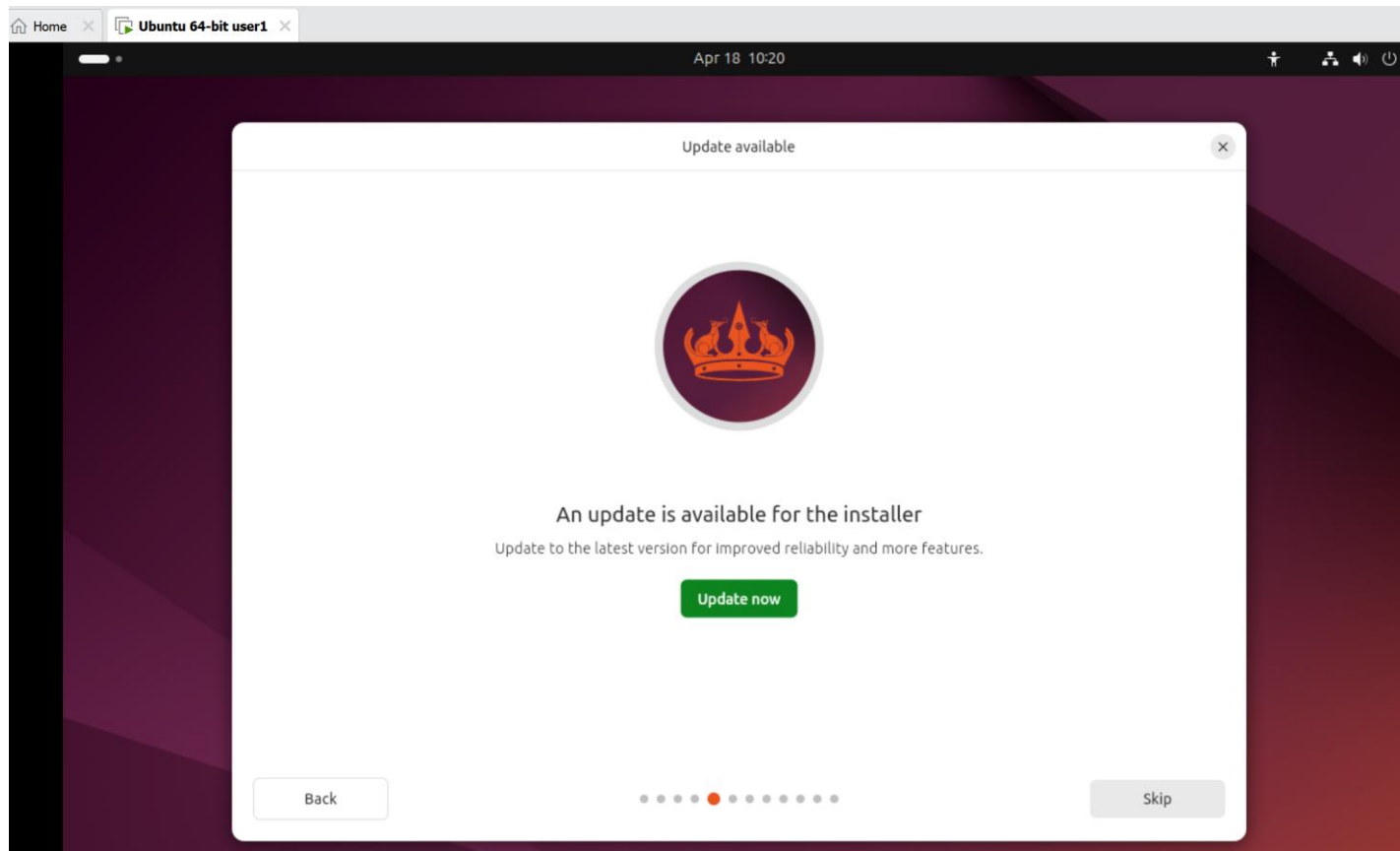
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



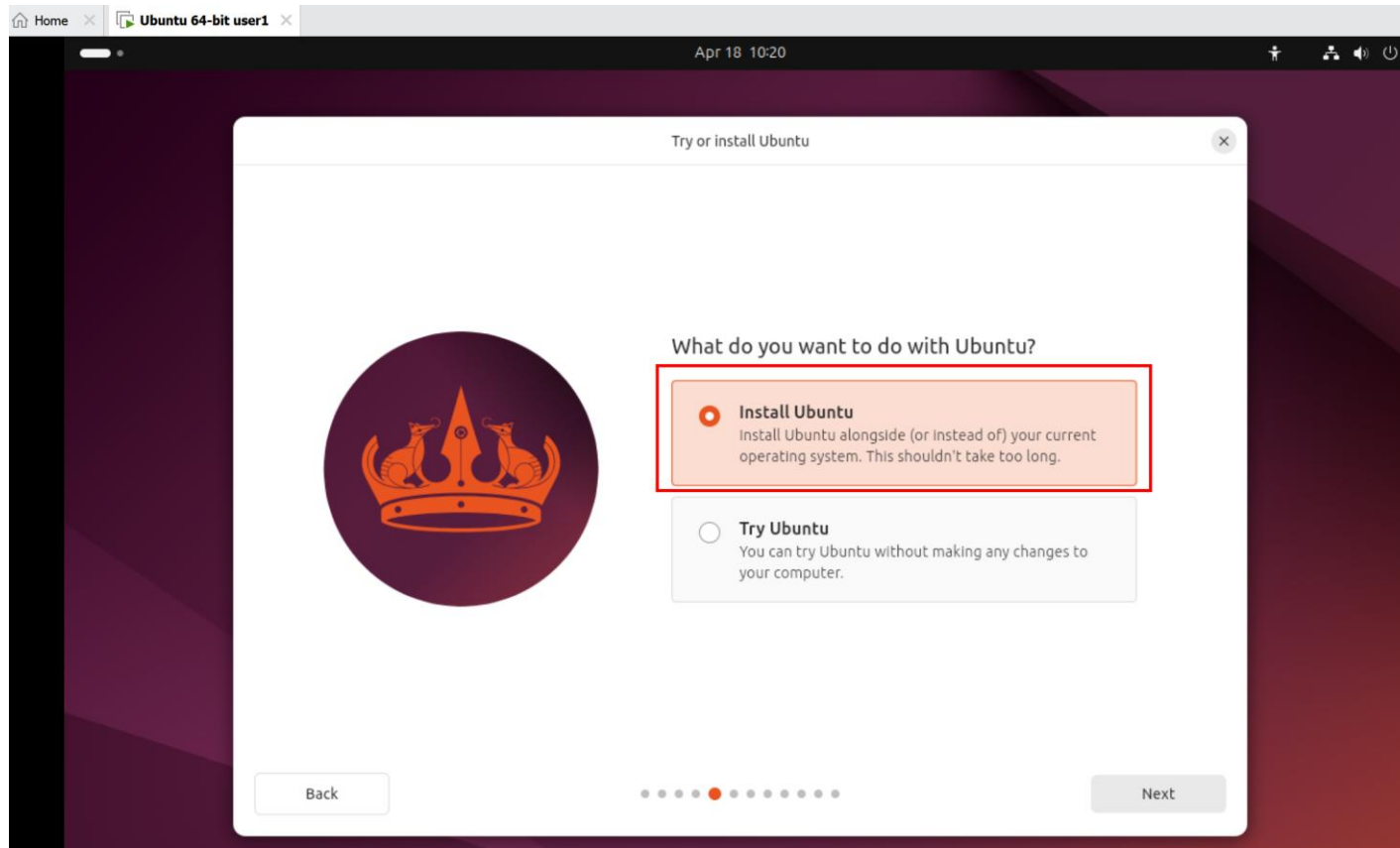
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



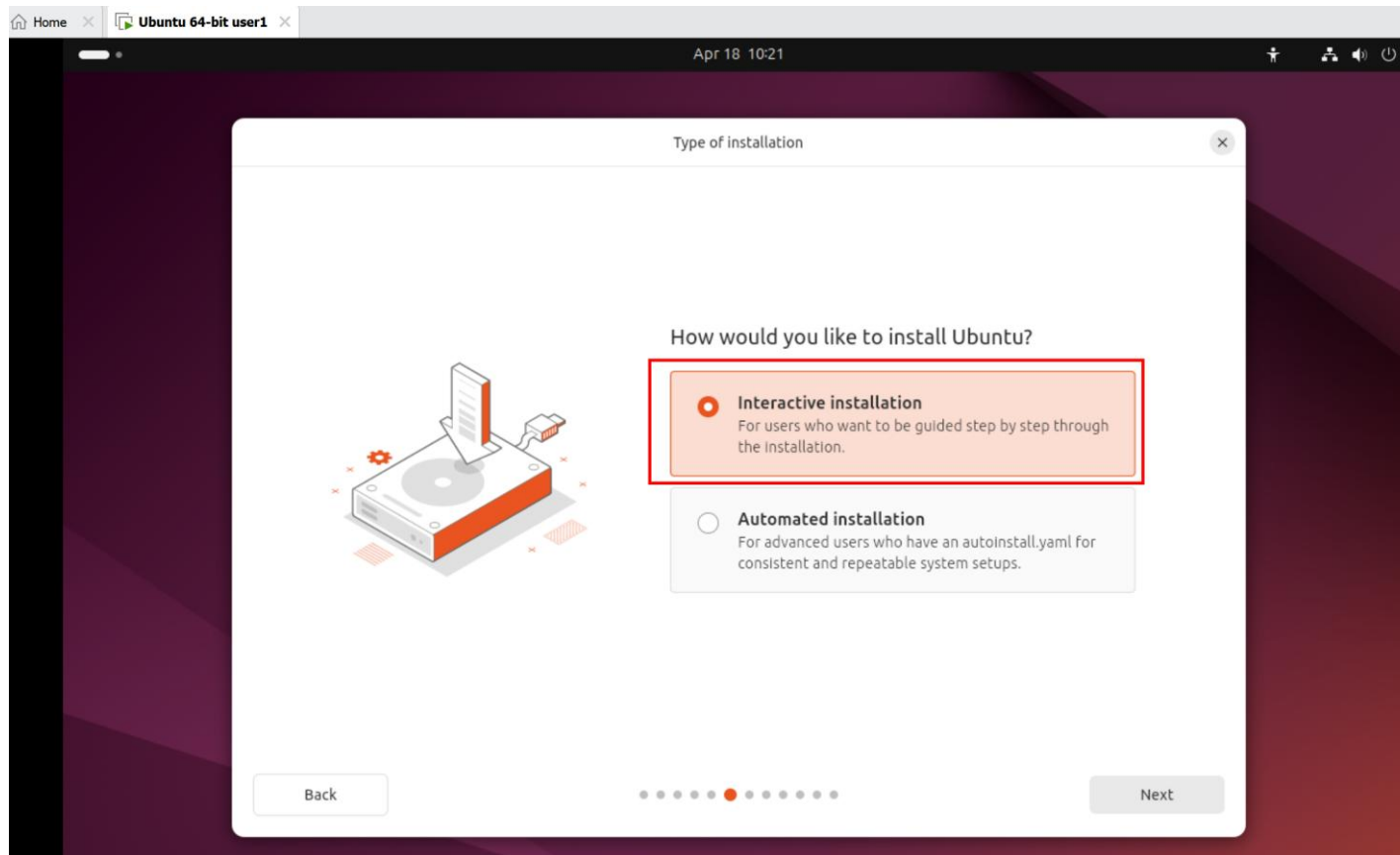
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



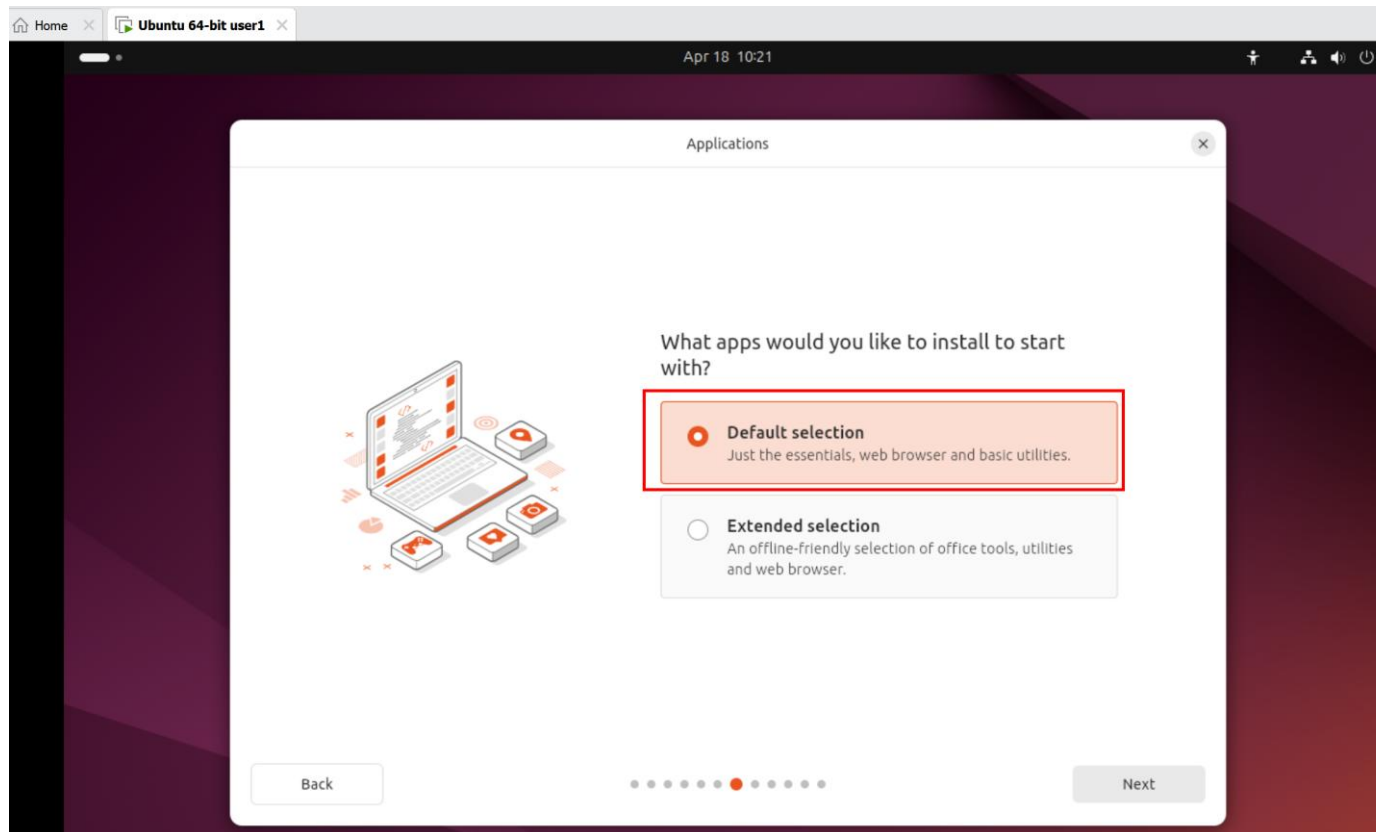
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



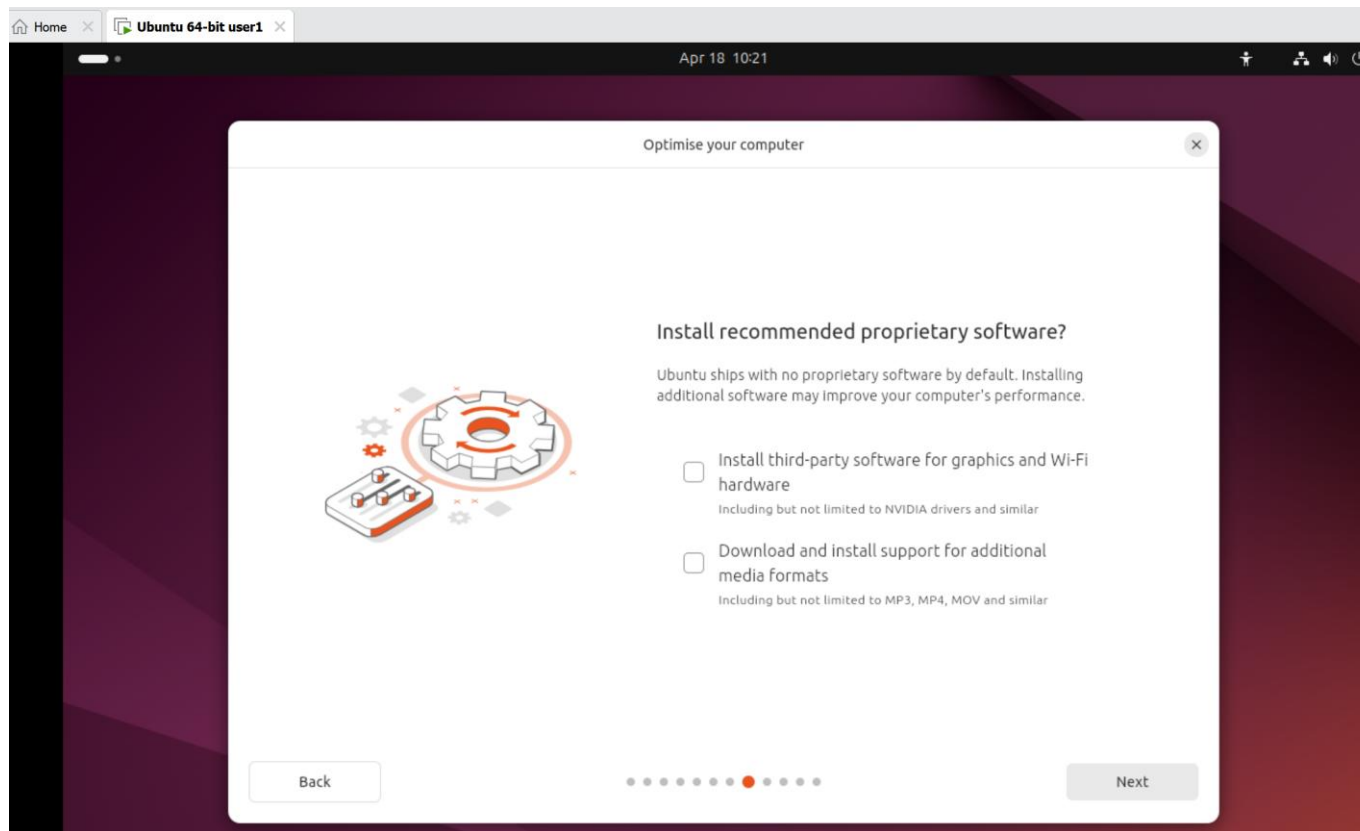
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



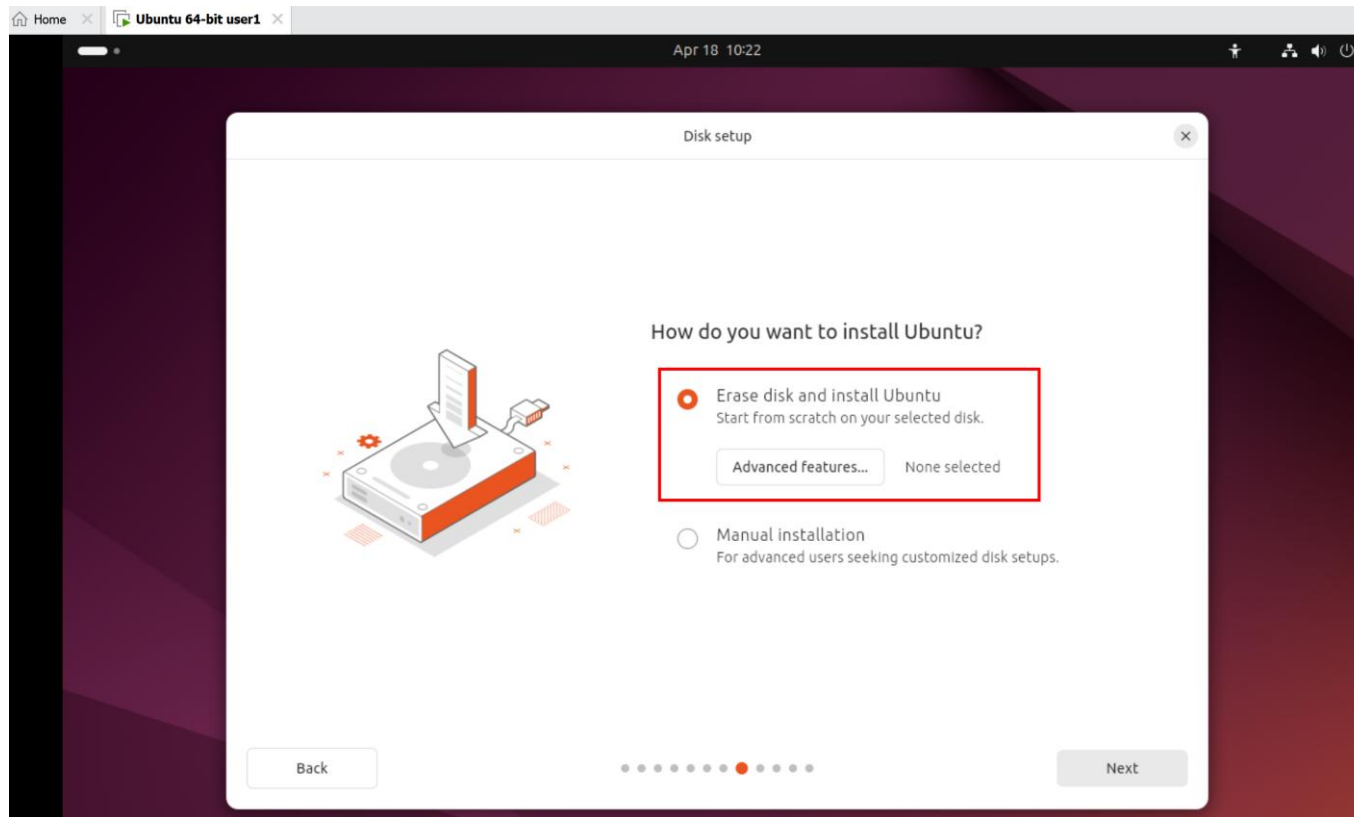
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



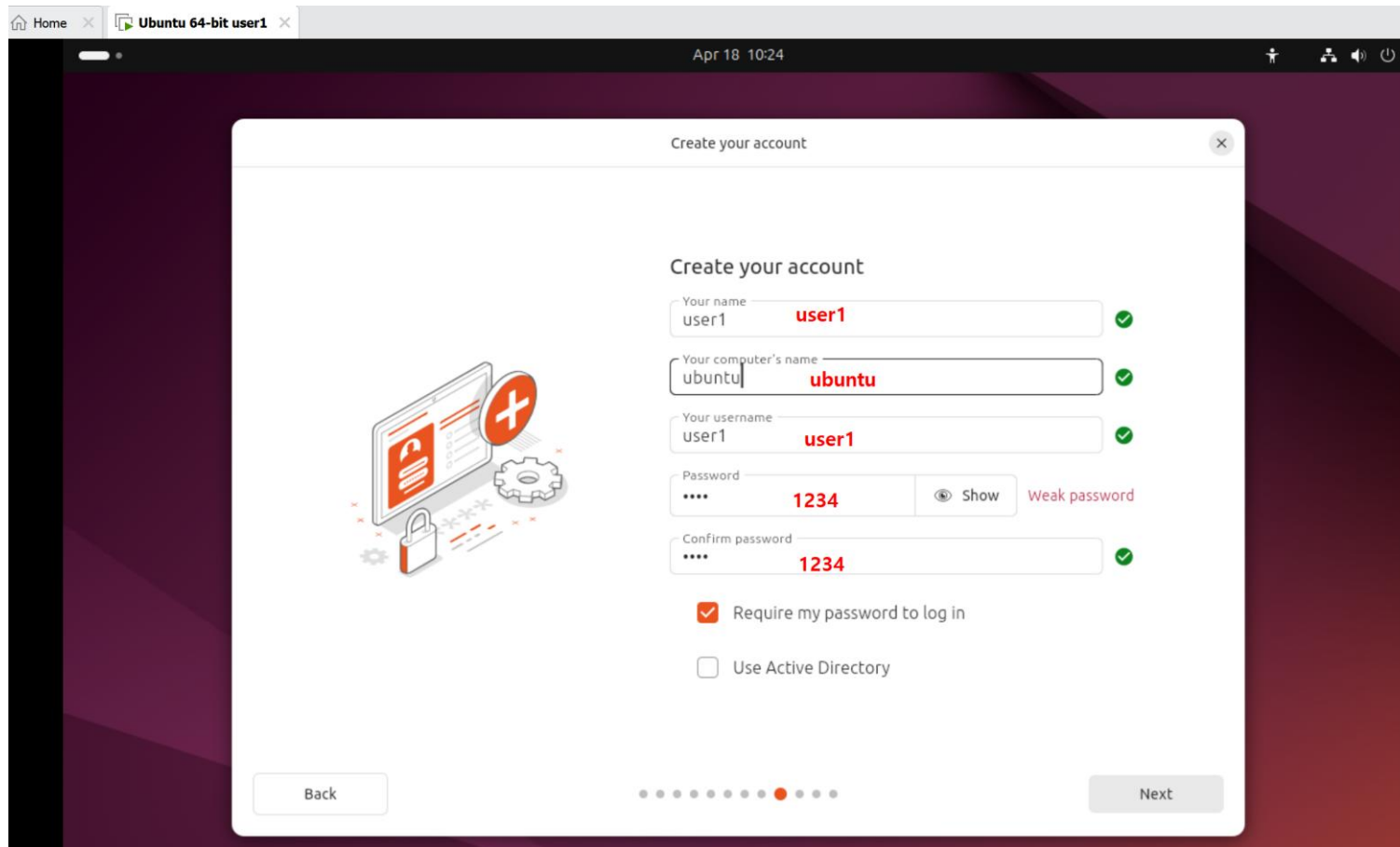
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



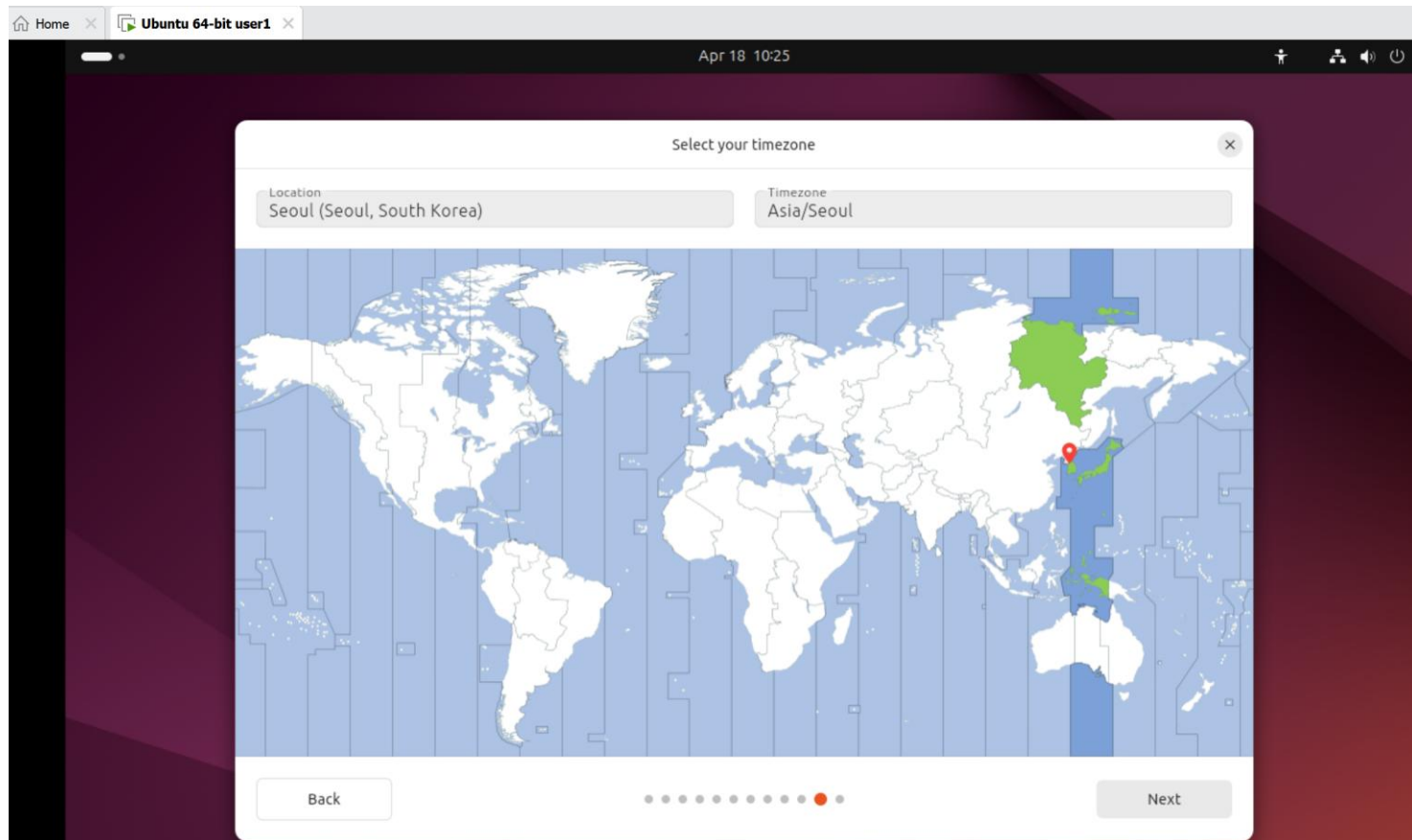
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치 (user1 계정 생성)



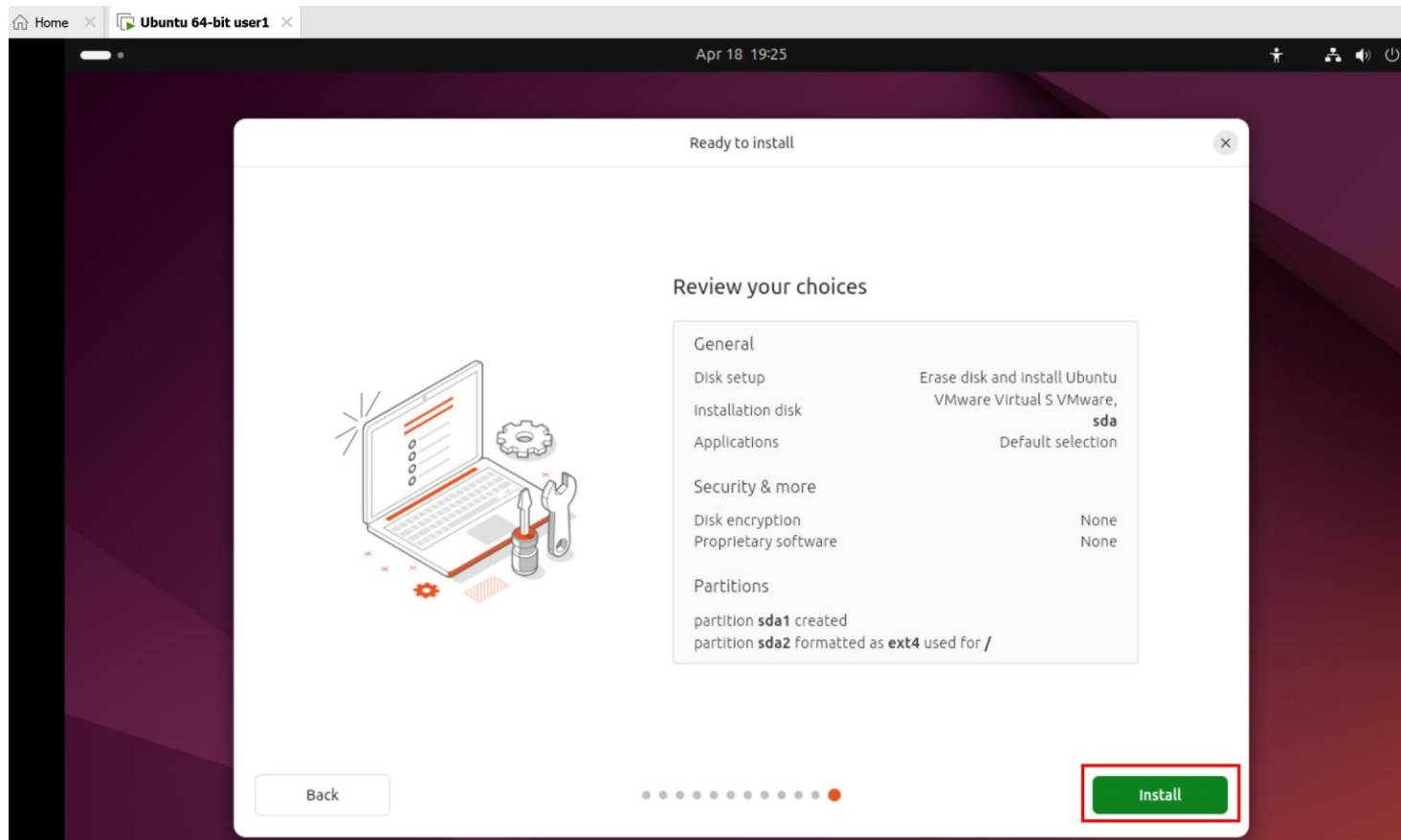
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



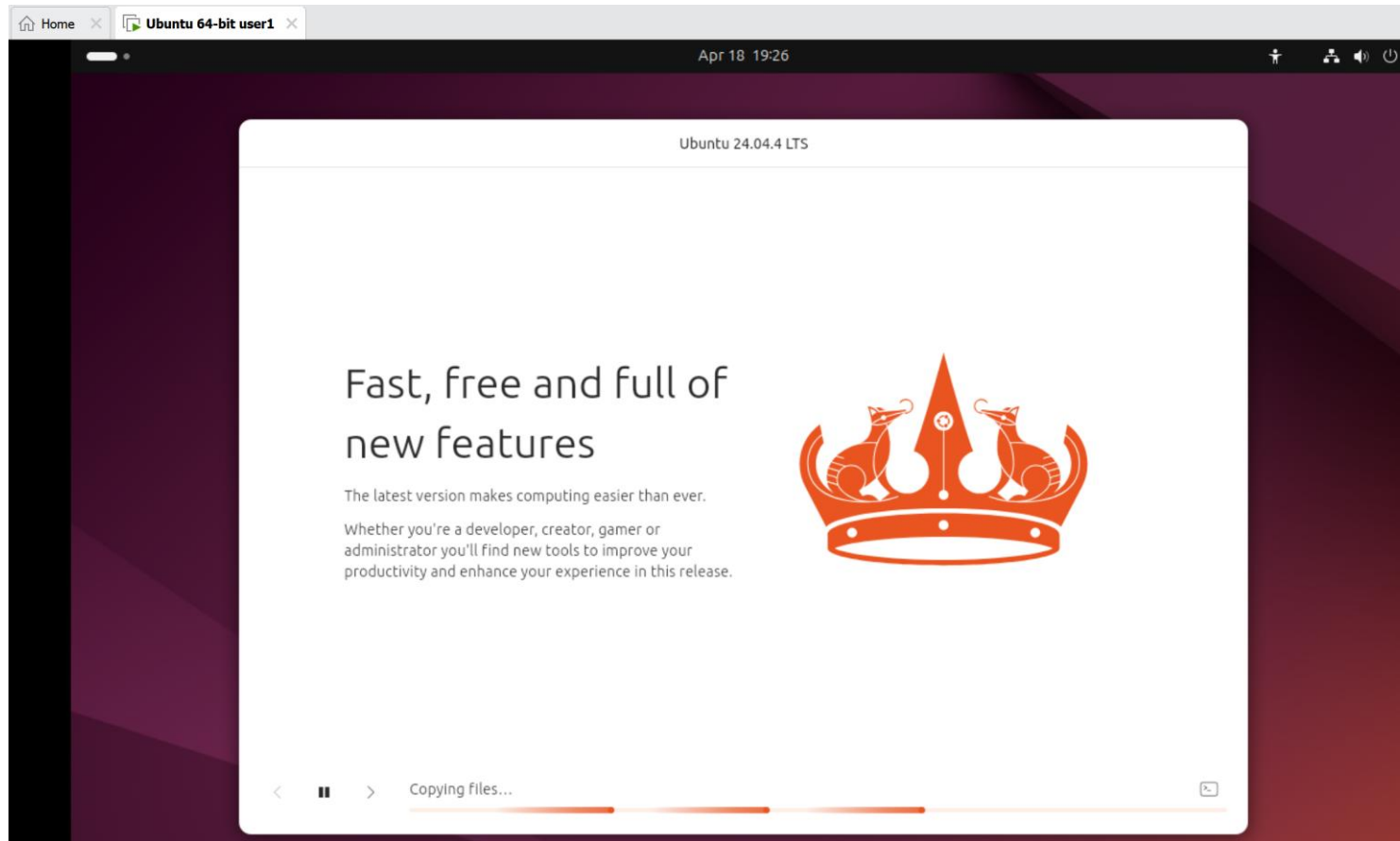
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



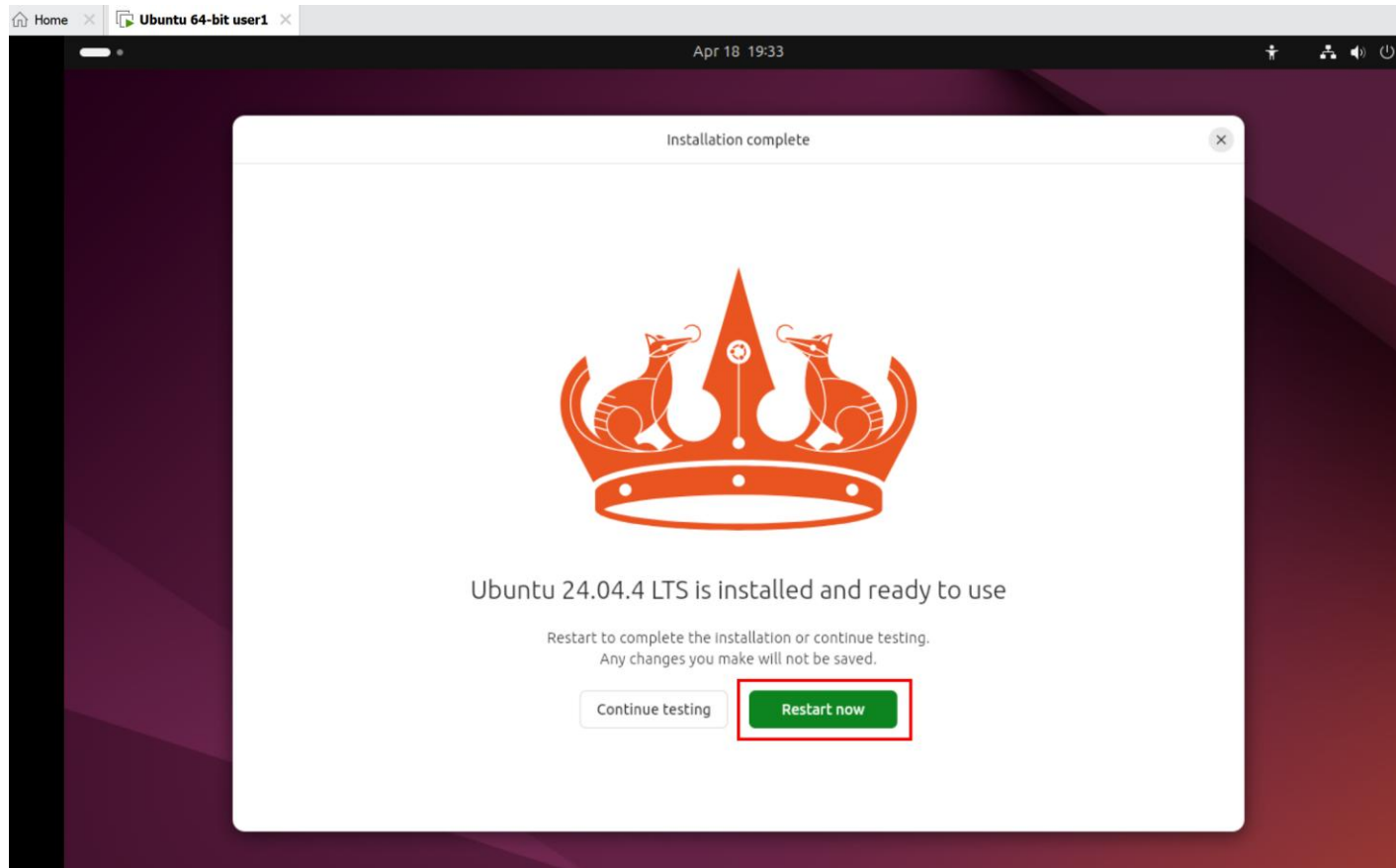
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



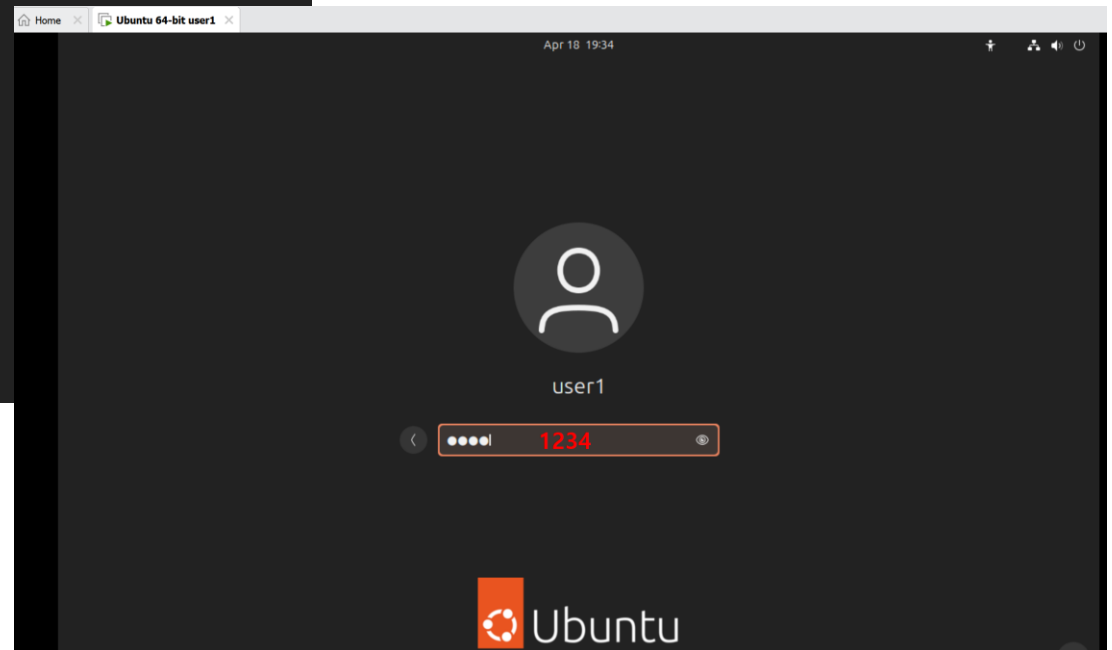
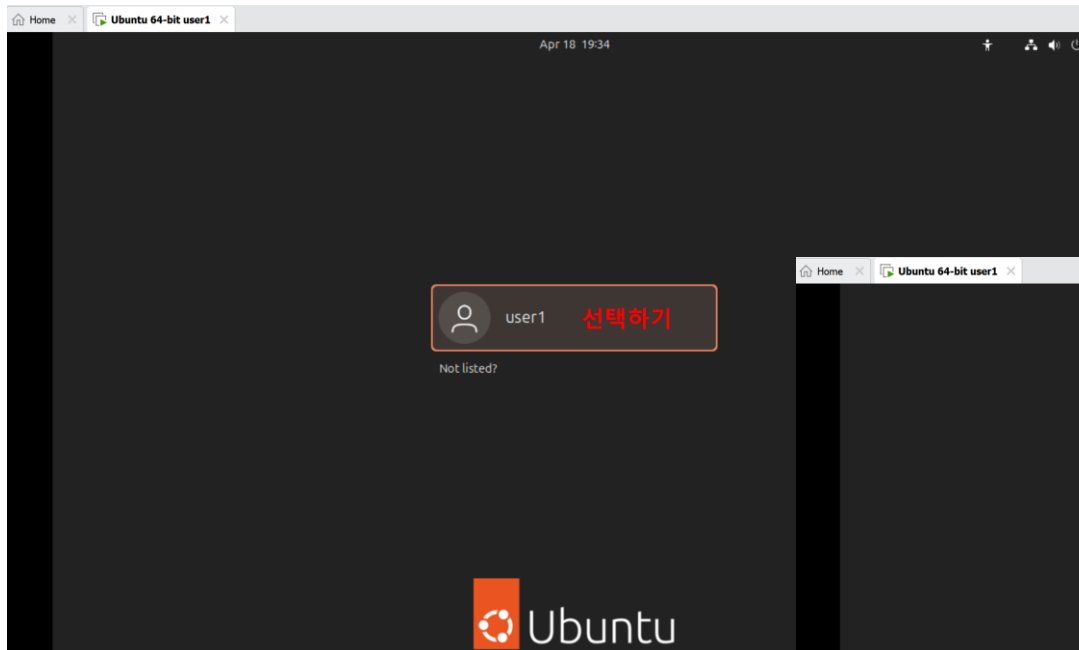
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치 (설치 완료)



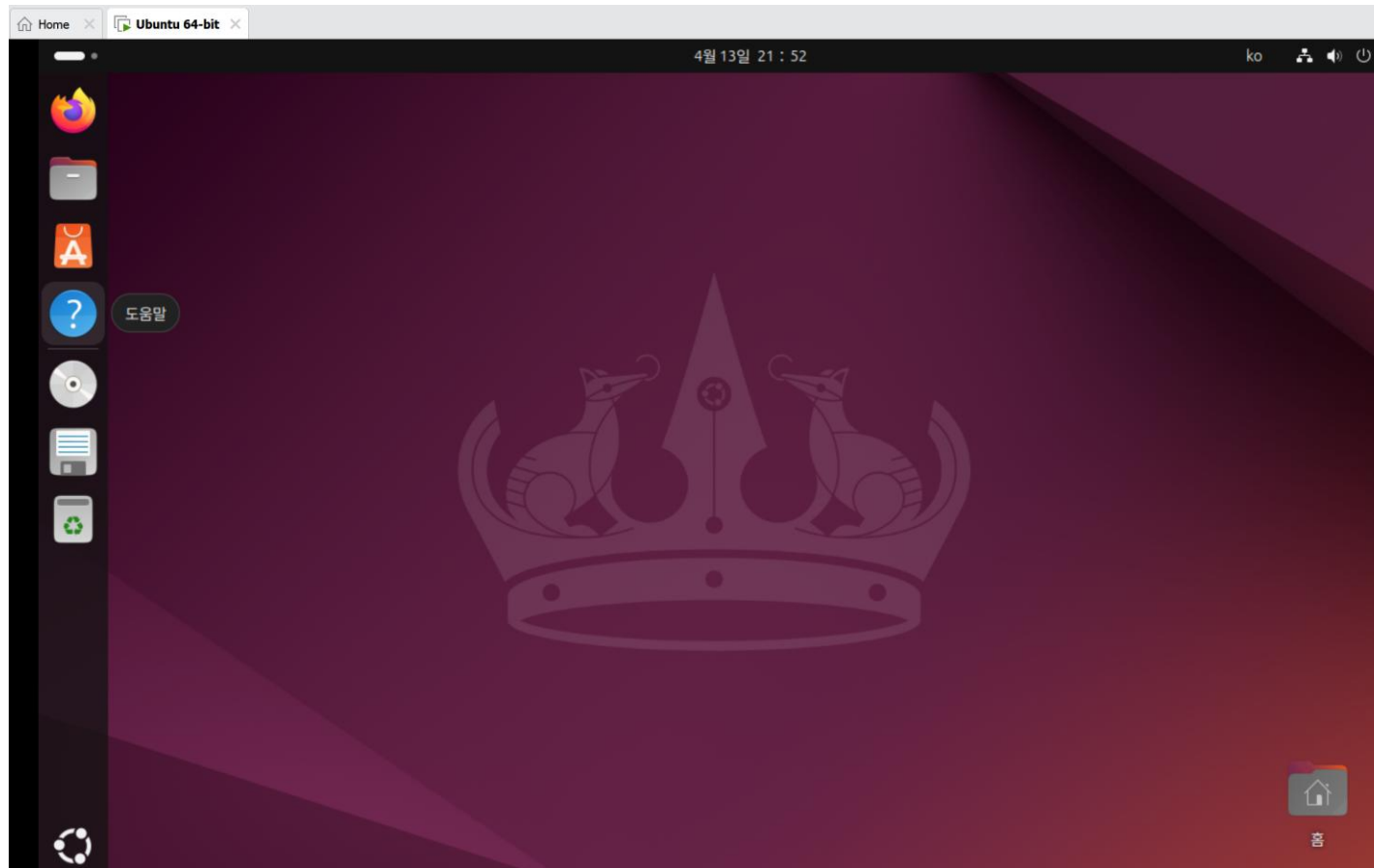
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



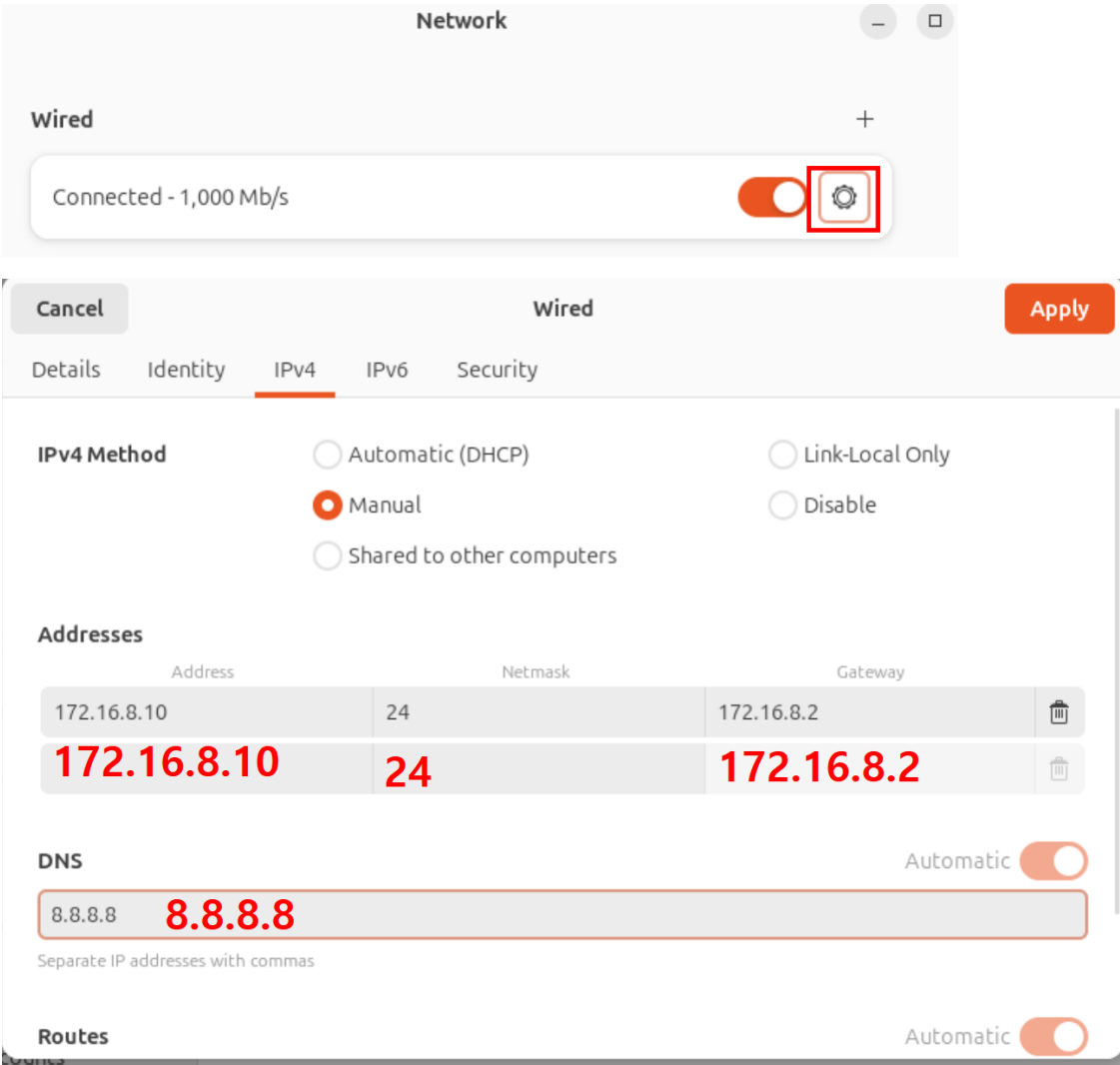
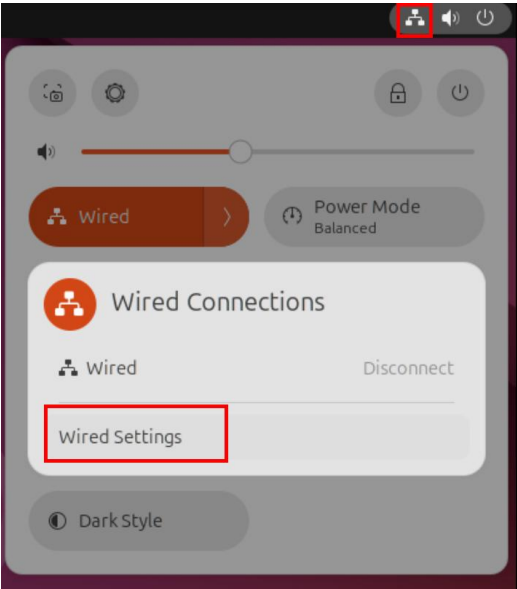
1.2 실습 환경 구성

VMWare 에 Ubuntu 설치



1.2 실습 환경 구성

Ubuntu 고정 IP 설정



1.2 실습 환경 구성

모든 응용 프로그램 업데이트

- 터미널 열기
- `sudo apt update`
- `sudo apt full-upgrade`

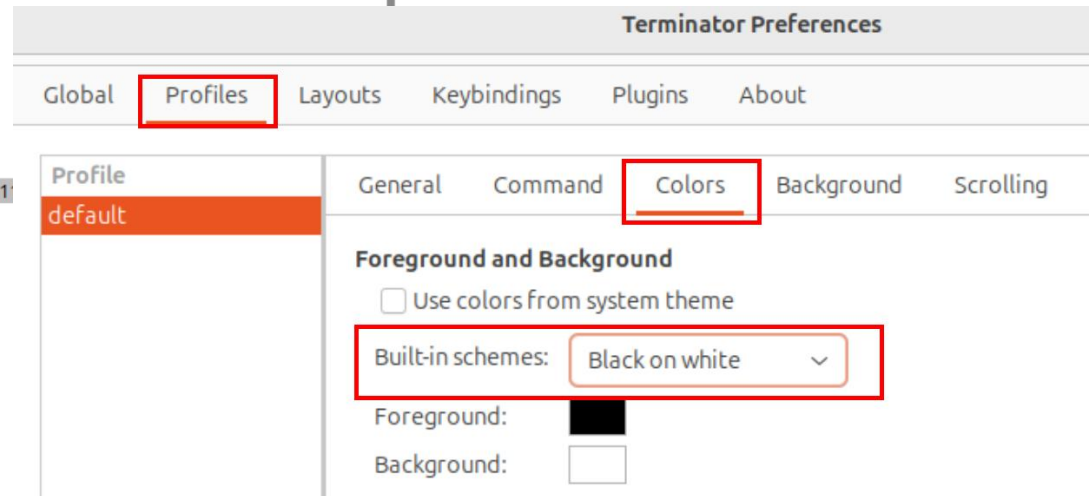
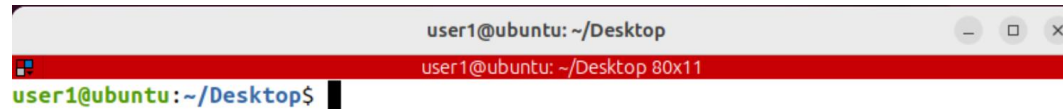
```
user1@ubuntu:~$ sudo apt update
[sudo] password for user1: 1234
Hit:1 http://kr.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://kr.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://kr.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
87 packages can be upgraded. Run 'apt list --upgradable' to see them.
user1@ubuntu:~$
```

```
user1@ubuntu:~$ sudo apt full-upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
```

1.2 실습 환경 구성

새로운 터미널 설치하기

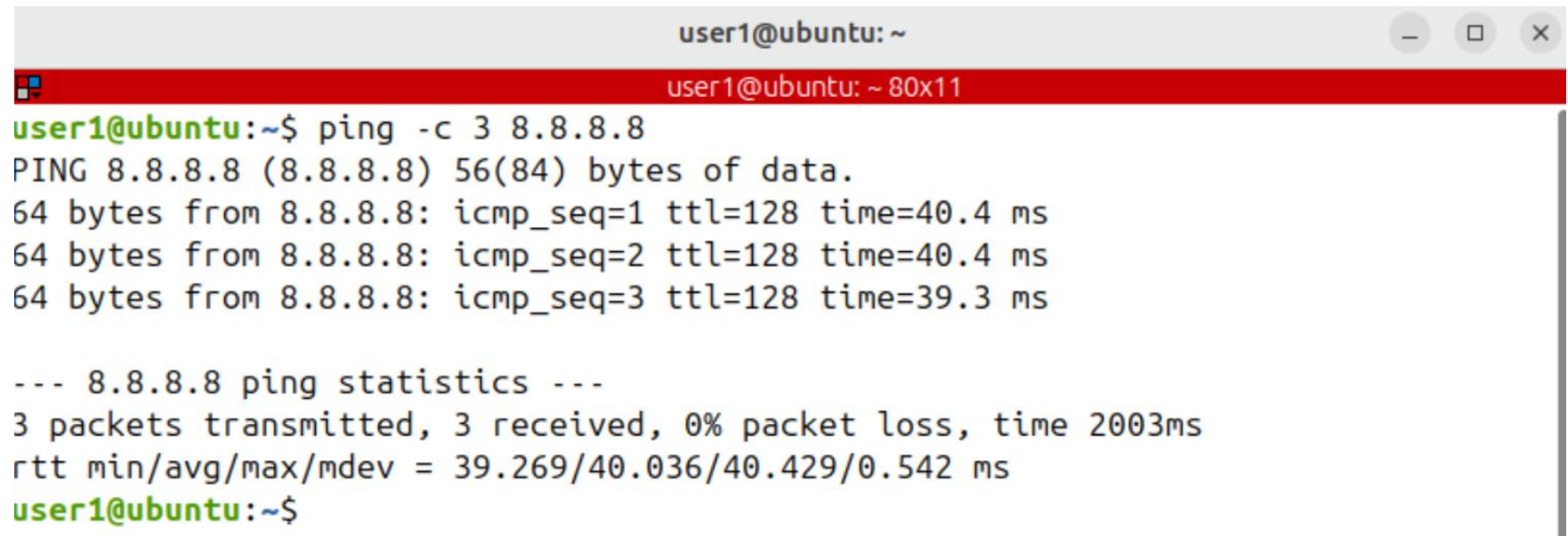
- `sudo apt update & sudo apt install terminator`



1.2 실습 환경 구성

인터넷 정상 동작 확인

- `ping -c 3 8.8.8.8`

A terminal window titled 'user1@ubuntu: ~' with standard Ubuntu window controls. The prompt is 'user1@ubuntu: ~ 80x11'. The user has entered the command 'ping -c 3 8.8.8.8'. The output shows three successful ping requests to 8.8.8.8 with varying response times (40.4 ms, 40.4 ms, 39.3 ms). A summary line indicates 0% packet loss and an average round-trip time of 40.036 ms.

```
user1@ubuntu:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=40.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=40.4 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=39.3 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 39.269/40.036/40.429/0.542 ms
user1@ubuntu:~$
```

1.2 실습 환경 구성

ubuntu에 SSH 설치

- Secure Shell 로 알려져 있음
- 시스템 관리자가 보안이 취약한 네트워크 환경에서 원격 시스템을 안전하게 구성하기 위해 사용함
- 기본 포트 번호: 22

- `sudo apt update & sudo apt install openssh-server`
- `systemctl status ssh`
- `sudo systemctl start ssh`
- `sudo systemctl enable ssh`

1.2 실습 환경 구성

MobaXterm Linux 원격접속 App 설치

<https://mobaxterm.mobatek.net/download.html>

The screenshot shows the MobaXterm website with a navigation bar at the top. Below the navigation bar, there are two main pricing sections: 'Home Edition' and 'Professional Edition'.

Home Edition is labeled 'Free' and lists features such as Full X server and SSH support, Remote desktop (RDP, VNC, Xdmcp), Remote terminal (SSH, telnet, rlogin, Mosh), X11-Forwarding, Automatic SFTP browser, Master password protection, Plugins support, Portable and installer versions, Full documentation, Max. 12 sessions, Max. 2 SSH tunnels, Max. 4 macros, and Max. 360 seconds for Tftp, Nfs and Cron. A 'Download now' button is highlighted with a red box.

Professional Edition is priced at '\$69 / 49€ per user*' and lists features such as Every feature from Home Edition +, Customize your startup message and logo, Modify your profile script, Remove unwanted games, screensaver or tools, Unlimited number of sessions, Unlimited number of tunnels and macros, Unlimited run time for network daemons, Enhanced security settings, 12-months updates included, Deployment inside company, and Lifetime right to use. A 'Subscribe online / Get a quote' button is visible at the bottom.

The screenshot shows the 'MobaXterm Home Edition' download page. It includes a section for downloading the current version, with two options: 'MobaXterm Home Edition v26.3 (Portable edition)' and 'MobaXterm Home Edition v26.3 (Installer edition)'. The portable edition button is highlighted with a red box.

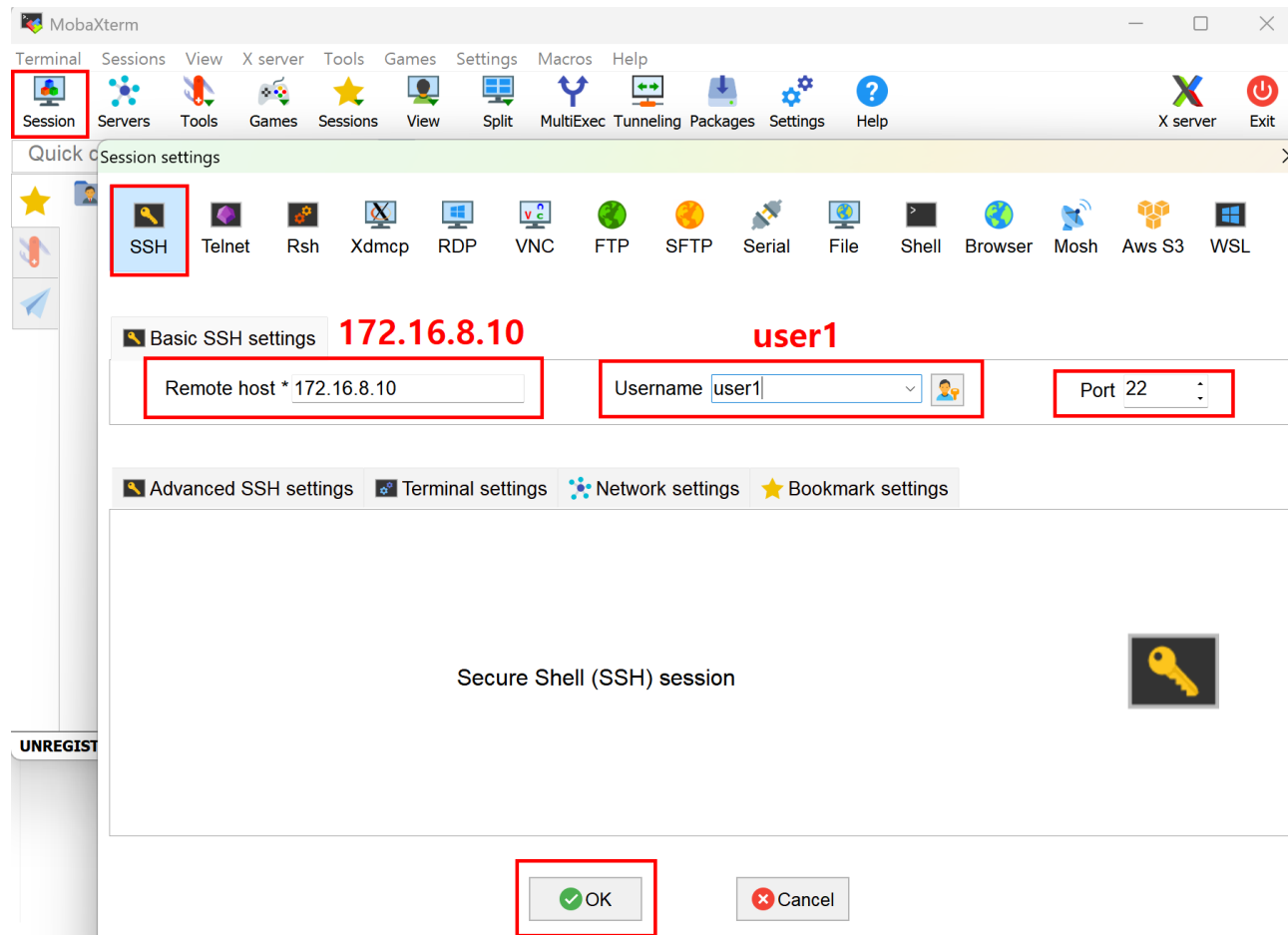
Below the download options, there is a list of files to be downloaded or installed, each with a corresponding icon:

- MobaXterm.ini
- MobaXterm backup.zip
- CygUtils.plugin
- CygUtils64.plugin
- MobaXterm_Personal_26.3.exe

The 'MobaXterm_Personal_26.3.exe' file is highlighted with a red box.

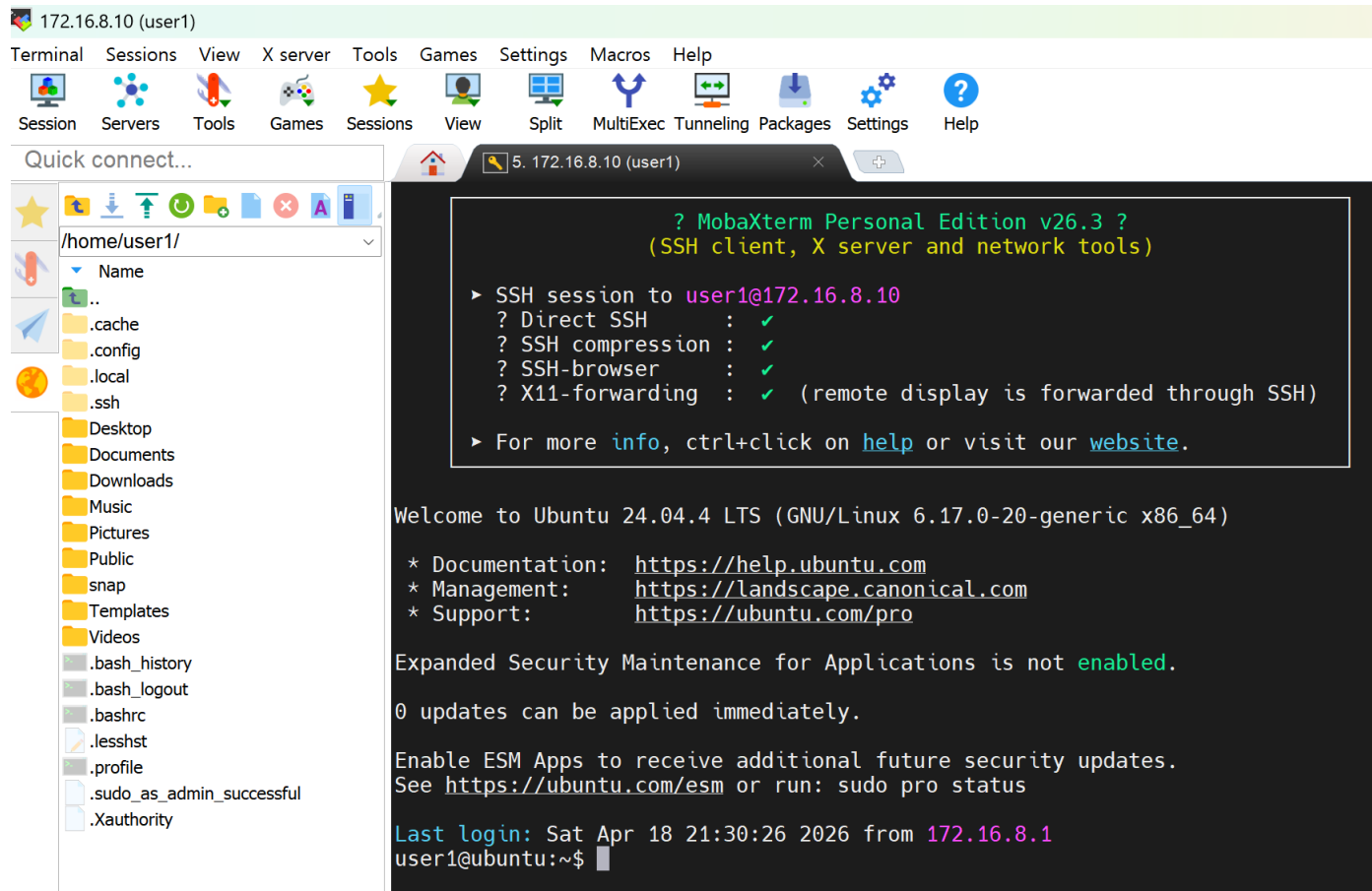
1.2 실습 환경 구성

MobaXterm 으로 원격접속 하기



1.2 실습 환경 구성

MobaXterm 으로 원격접속 하기



1.2 실습 환경 구성

pem 파일 이용한 Linux에 로그인하기

공개키(public key) 와 개인키 (private key) 생성하기

```
ssh-keygen -q -N "" -f test_key.pem
```

공개키의 내용을 authorized_keys 파일에 기록한다

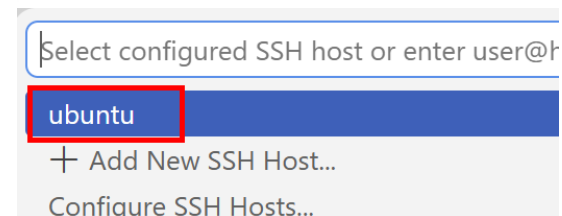
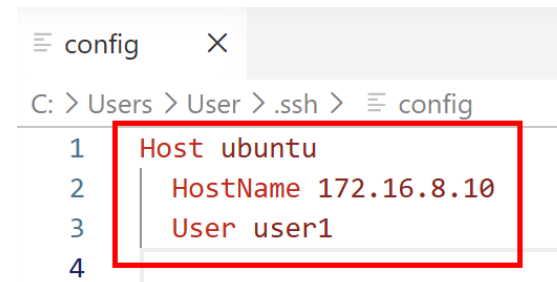
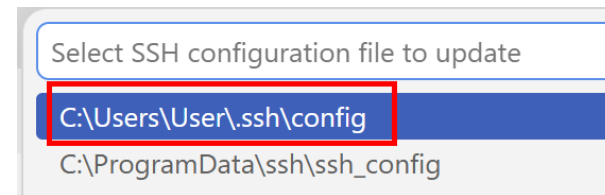
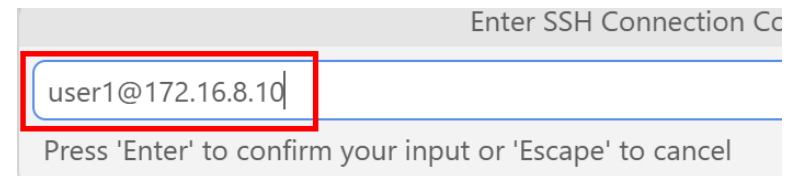
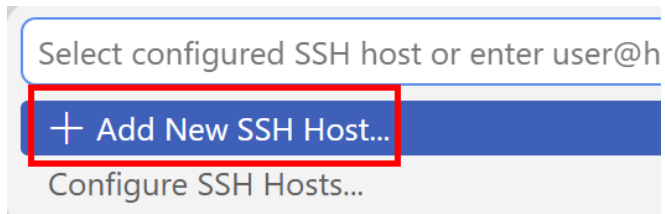
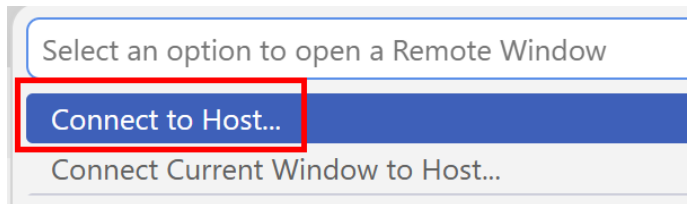
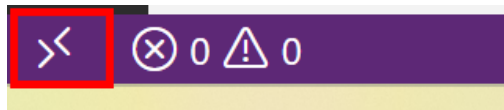
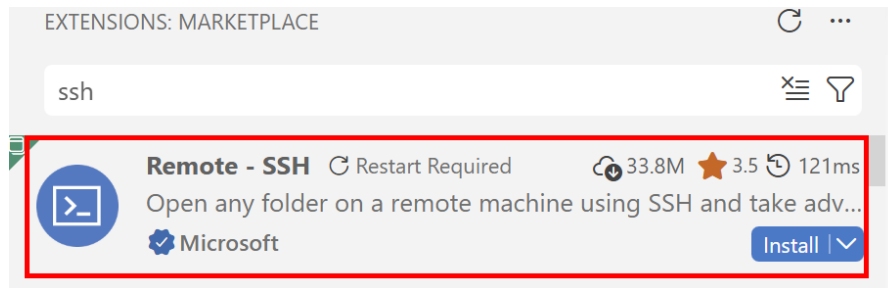
```
cat test_key.pem.pub >> ~/.ssh/authorized_keys
```

권한 낮추기

```
chmod 600 ~/.ssh/authorized_keys
```

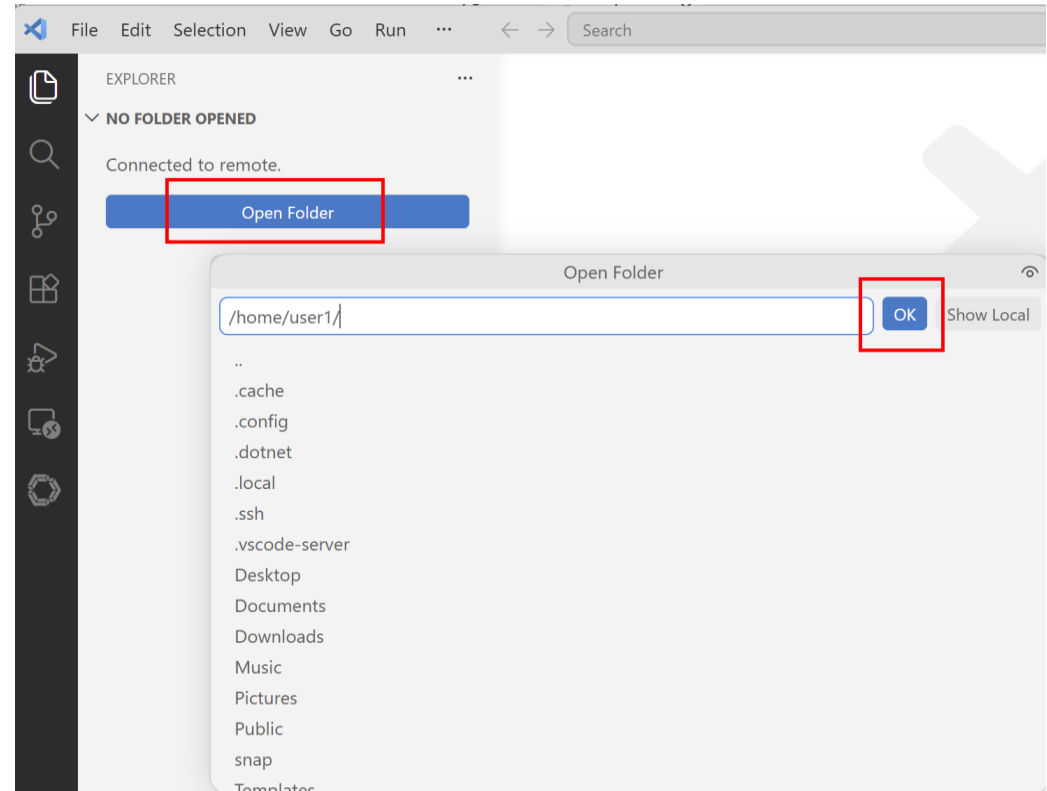
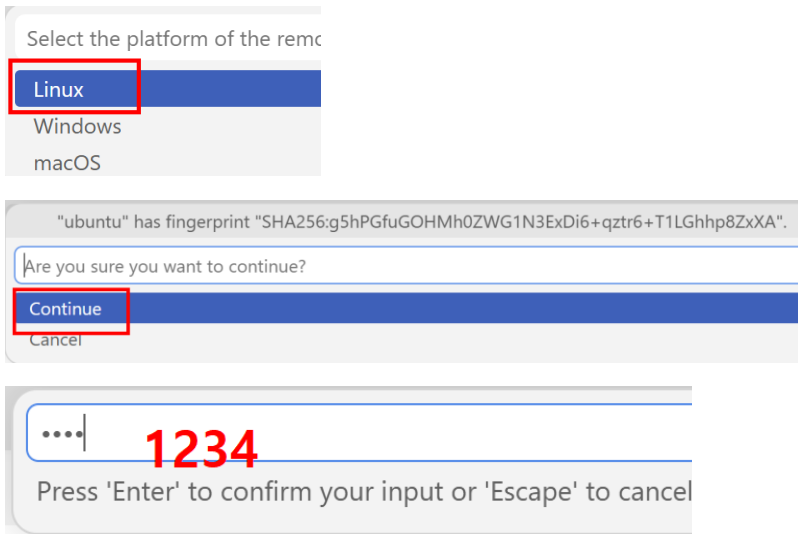
1.2 실습 환경 구성

VSC에서 Linux에 로그인하기



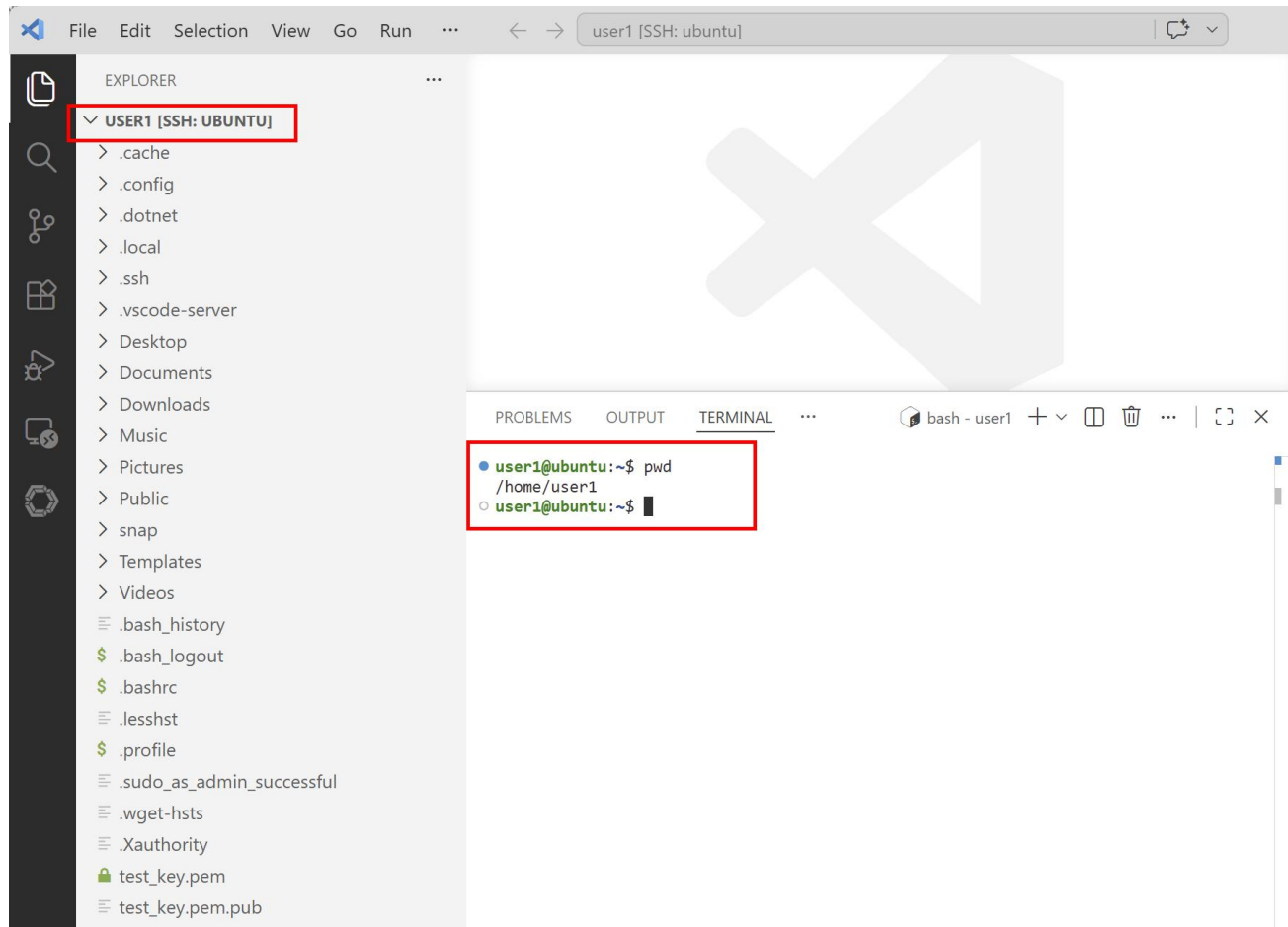
1.2 실습 환경 구성

VSC에서 Linux에 로그인하기



1.2 실습 환경 구성

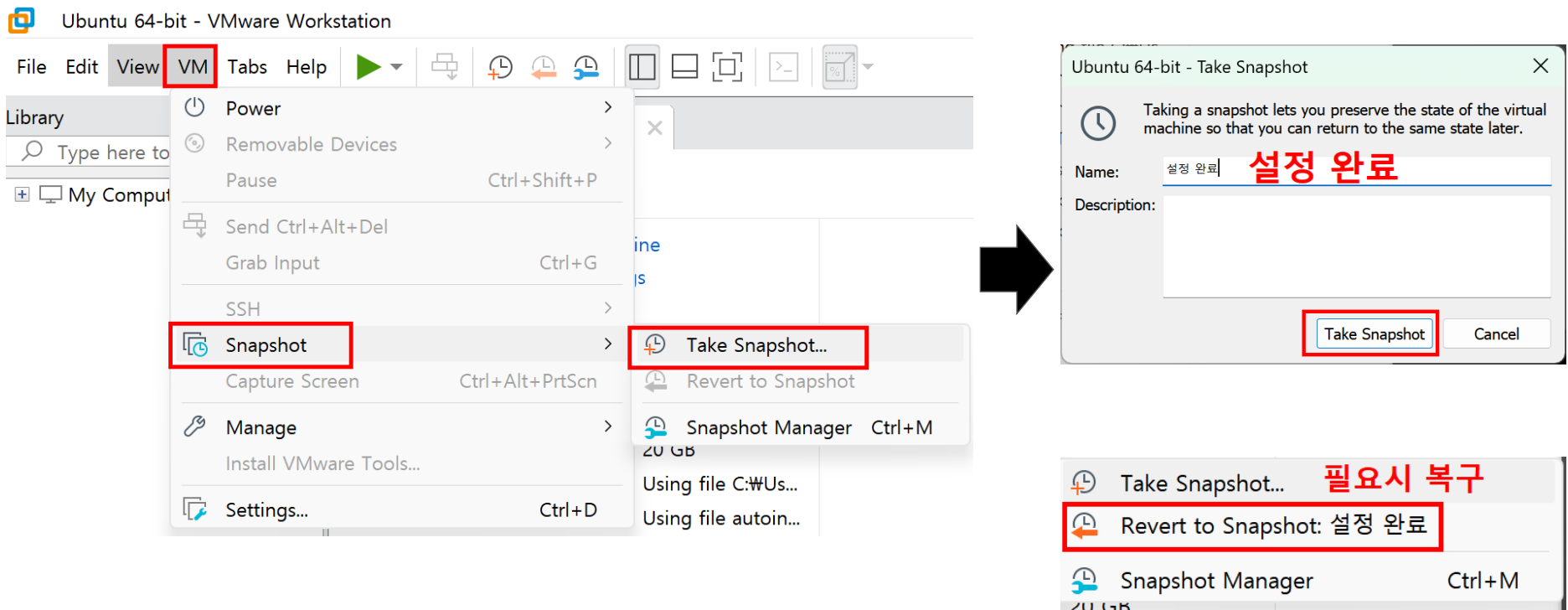
VSC에서 Linux에 로그인하기



1.2 실습 환경 구성

스냅샷 만들기

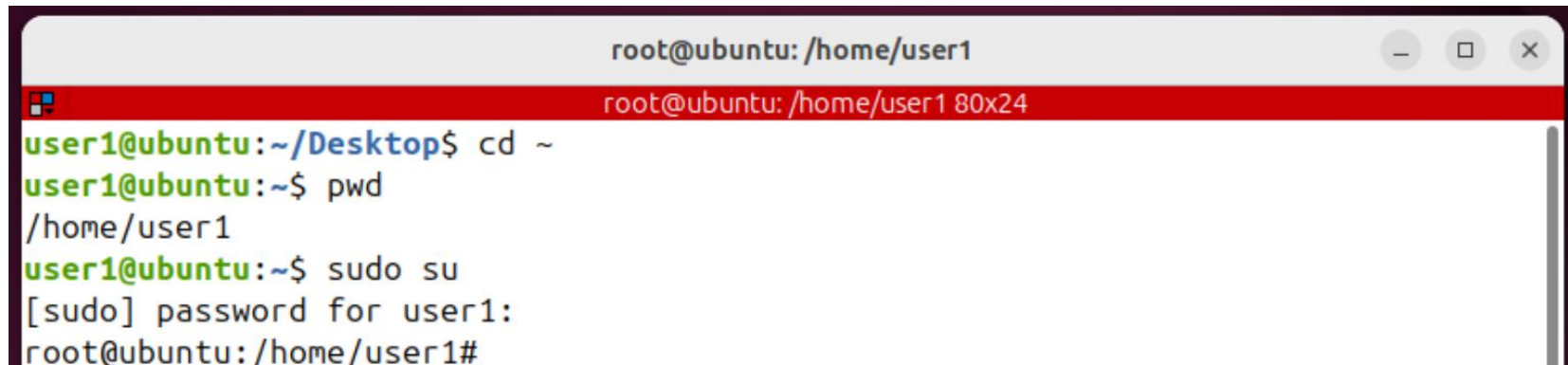
1. 먼저 ubuntu 종료
2. VMWare > VM 메뉴 > Snapshot > Take Snapshot... 선택



1.3 프롬프트 기호와 홈 디렉터리

프롬프트와 홈 디렉터리

- 사용자의 명령 입력을 기다리는 표시
- 셸(shell)에 따라 다르게 표시됨.
- bash 셸의 경우 \$ 로 표시. 관리자의 경우는 #로 표시
- 홈 디렉터리로 가는 방법: `cd ~`



```
root@ubuntu: /home/user1
user1@ubuntu: ~/Desktop$ cd ~
user1@ubuntu: ~$ pwd
/home/user1
user1@ubuntu: ~$ sudo su
[sudo] password for user1:
root@ubuntu: /home/user1#
```

2.1 리눅스 명령어

명령어 도움말 지원

- man 명령어
ex. \$ man ls
- 명령어 --help
ex. \$ ls --help

2.2 터미널 명령어

Tab 키 활용

- 자동완성 기능
- 파일이름이나 디렉터리 이름을 자동완성 지원
ex. `$ cat /etc/passwd`

터미널 단축키

- 화면 클리어 : `Ctrl + L`
- 터미널 닫기 : `Ctrl + D`
- 터미널 열기: `Ctrl + Alt + T`
- 명령어 첫 글자 및 끝 글자로 가기 : `Ctrl + A` , `Ctrl + E`
- 현재 명령어 삭제 : `Ctrl + U`
- 현재 진행중인 작업 취소: `Ctrl + C`

2.2 터미널 명령어

명령어 history

- 터미널에서 사용했던 명령어들은 사용자의 홈 디렉터리에 저장됨
- 각 사용자가 실행한 명령어들의 이력을 확인할 수 있음

ex. history

cd

cat .bash_history

명령어 정리

- !번호: <= 지정된 번호 명령어 실행
- !! <= 방금 실행한 명령어 다시 실행하기
- !일부글자 <= history에서 보여준 명령어들의 일부 글자만 지정해서 실행하기
- !일부글자:p <= 실행하기 전에 실행할 명령어인지 확인하기
- history -c <= 세션에 저장된 history 전체 삭제

2.3 root 권한으로 실행

리눅스 사용자 종류

- 권한이 없는 사용자 (non-privileged users)
 - 사용자 홈 디렉터리에서만 작업이 가능
- root 사용자 (superuser 또는 administrator)
 - 어떤 명령도 실행이 가능
 - 어떤 파일도 접근이 가능
 - 사실 Linux 시스템에서 모든 것을 할 수 있음

일반 사용자가 root 권한으로 작업하는 방법

- `$ sudo su` <= root 권한을 얻고 명령어 실행 (`sudo -s` 동일한 기능)
- `$ sudo su -` <=root 권한을 얻고 root 홈 디렉터리로 바로 가서 명령어 실행
- `$ sudo 명령어` <=root 계정에 로그인 없이 일반사용자 계정에서 명령어 실행
- `$ sudo passwd root` <= root 계정에 암호를 설정하고 root로 로그인해서 명령어 실행

3.1 리눅스 파일 시스템

개념

- 데이터가 저장되고 검색되는 것을 관리.
- Linux 시스템에서는 모든 것을 파일로 간주함 (USB 같은 물리적인 스틱 포함)
- 디렉터리도 특별한 종류의 파일로 간주함.

파일 종류

- 일반 파일 (regular file)
 - 데이터를 저장하는데 주로 사용
 - 각종 텍스트 파일, 실행파일, 이미지 파일 등 리눅스에서 사용하는 대부분의 파일에 해당됨
- 디렉터리 (directory)
 - Linux에서는 디렉터리도 파일로 취급
- 심볼릭 링크
 - 원본 파일을 대신하여 다른 이름으로 파일명을 지정한 것 (윈도의 바로가기와 비슷)
- 장치 파일
 - Linux에서는 하드디스크나 USB 같은 각종 장치도 파일로 취급

3.1 리눅스 파일 시스템

파일 종류 확인

- file 명령어 이용

ex. \$ file /etc/passwd

\$ file /etc

\$ file /usr/bin/ls

- ls -F

-F 옵션은 파일의 종류를 구분하여 표시

/: 디렉터리

@: 심볼릭 링크

*: 실행파일

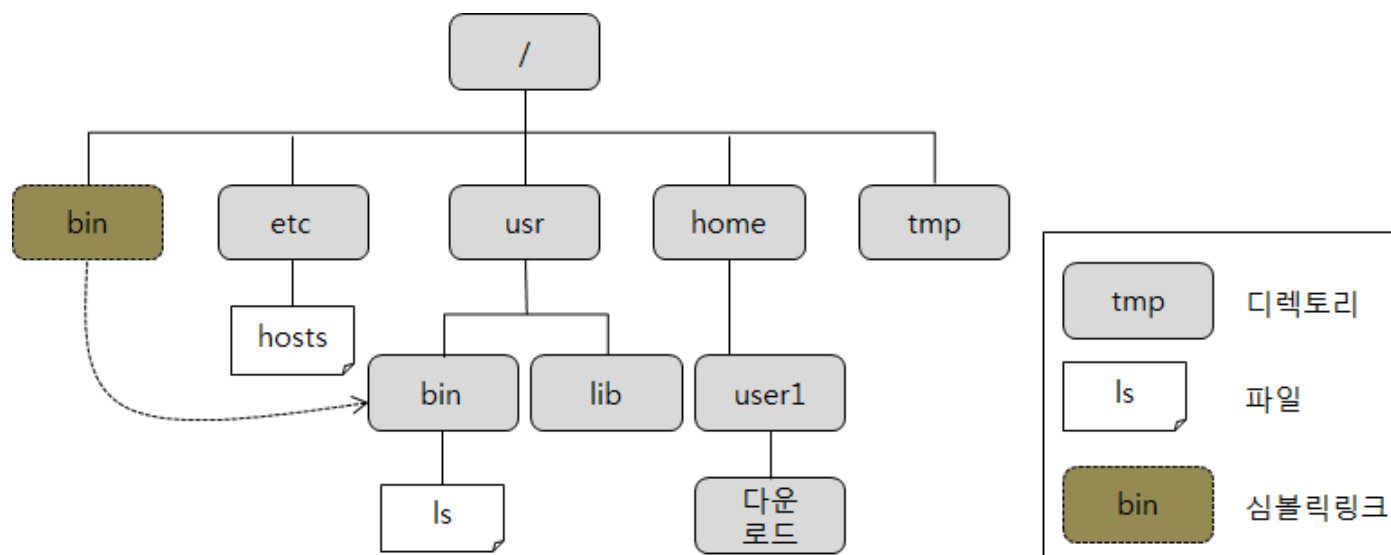
이외: 일반파일

ex. ls -aF /etc

3.1 리눅스 파일 시스템

디렉터리 계층 구조

- 리눅스에서는 파일을 효율적으로 관리하기 위해서 디렉터리를 계층적으로 구성 (트리 구조)
- 모든 디렉터리의 출발점은 루트(root) 디렉터리이며 / 로 표시함.



3.1 리눅스 파일 시스템

디렉터리 종류와 특징

디렉터리	기능
dev	장치 파일이 담긴 디렉터리이다.
home	사용자 홈 디렉터리가 생성되는 디렉터리이다.
media	시디롬이나 USB 같은 외부 장치를 연결(마운트라고 함)하는 디렉터리이다.
opt	추가 패키지가 설치되는 디렉터리이다.
root	root 계정의 홈 디렉터리이다. 루트(/) 디렉터리와 다른 것이므로 혼동하지 않도록 한다.
sys	리눅스 커널과 관련된 파일이 있는 디렉터리이다.
usr	기본 실행 파일과 라이브러리 파일, 헤더 파일 등 많은 파일이 있다. 참고로 usr은 Unix System Resource의 약자이다.
boot	부팅에 필요한 커널 파일을 가지고 있다.
etc	리눅스 설정을 위한 각종 파일을 가지고 있다.
lost+found	파일 시스템에 문제가 발생하여 복구할 경우, 문제가 되는 파일이 저장되는 디렉터리로 보통은 비어있다.
mnt	파일 시스템을 임시로 마운팅 하는 디렉터리이다.
proc	프로세스 정보 등 커널 관련 정보가 저장되는 디렉터리이다.
run	실행 중인 서비스와 관련된 파일이 저장된다.
srv	FTP나 Web 등 시스템에서 제공하는 서비스의 데이터가 저장된다.
tmp	시스템 사용 중에 발생하는 임시 데이터가 저장된다. 이 디렉터리에 있는 파일들은 재부팅 하면 모두 삭제된다.
var	시스템 운영 중에 발생하는 데이터나 로그 등이 저장되는 디렉터리이다.

3.1 리눅스 파일 시스템

홈 디렉터리 (home directory)

- 각 사용자에게 할당된 디렉터리로서 처음 사용자 계정을 만들 때 지정.
- 사용자는 자신의 홈 디렉터리 아래에 파일이나 서브 디렉터리를 생성하며 작업 가능.
- 홈 디렉터리는 ~ 로 표시함.

ex \$ cd ~

작업 디렉터리 (working directory)

- 현재 작업중인 디렉터리를 의미 (현재 디렉터리(current directory)라고도 부름)
- 현재 디렉터리는 . 로 표시함.
- 현재 디렉터리 위치는 pwd 명령으로 확인 가능

ex \$ pwd

3.2 파일 시스템 경로

경로명

- 파일 시스템에서 디렉터리 계층 구조에 있는 특정 파일이나 디렉터리의 위치 표시.
- 경로명에서 각 경로를 구분하는 구분자로 / 사용.
- 경로명에서 가장 앞에 있는 / 는 root 디렉터를 뜻하지만 경로명 중간에 있는 /는 구분자임.

절대 경로 (absolute path)

- 항상 root(/) 디렉터리로부터 시작.
- / 디렉터리부터 시작하여 특정 파일이나 디렉터리의 위치까지 이동하면서 거치게 되는 모든 중간 디렉터리의 이름을 표시.
- 특정 위치를 가리키는 절대 경로명은 항상 동일함.

상대 경로 (relative path)

- 현재 디렉터를 기준으로 시작
- / 이외의 문자로 시작 ex. ./ 또는 ../
- 현재 디렉터를 기준으로 상위 디렉터리(..) 또는 서브 디렉터리로 경로 설정 가능.
- 상대 경로명은 현재 디렉터리가 어디냐에 따라 달라짐.

3.2 파일 시스템 경로

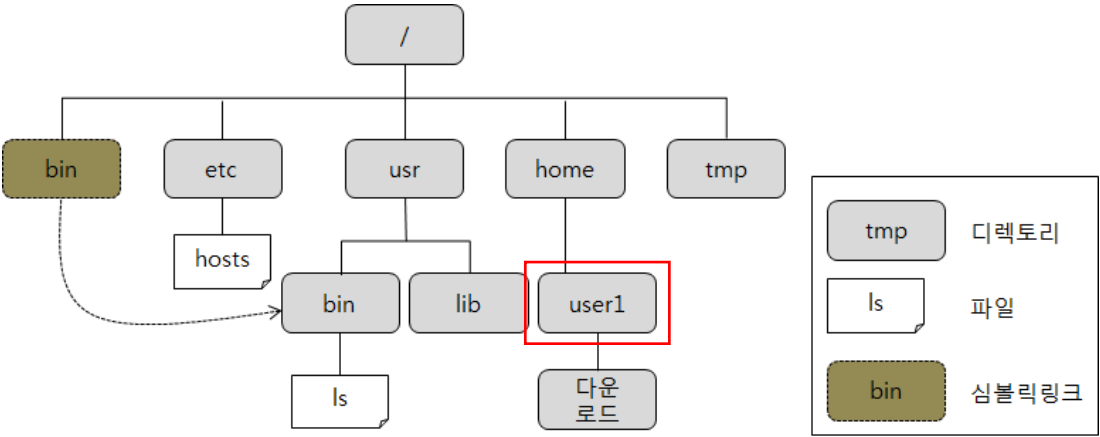
디렉터리 이동하기

- 문법: `cd [디렉터리]`
- 디렉터리 지정 시 절대경로/상대경로 모두 가능
ex. `cd /tmp`
`cd ../usr/lib`

홈 디렉터리로 이동

- `cd /home/user1` # 절대 경로
- `cd ~` # 홈 디렉터리를 나타내는 ~ 기호 사용
- `cd` # 목적지 디렉터리를 지정하지 않고 `cd` 명령만 사용해도 홈 디렉터리로 이동

3.2 파일 시스템 경로



현재 디렉터리가 user1일 때

- user1의 절대 경로명: /home/user1
- user1 아래 ‘다운로드’의 절대 경로명: /home/user1/다운로드
- ‘다운로드’의 상대 경로명: 다운로드 또는 ./다운로드
- hosts 파일의 상대 경로명: ../../etc/hosts

디렉터리/파일명	절대 경로	상대 경로
/		
home		
tmp		
lib		
ls		

3.3 디렉터리 내용 보기

디렉터리 내용 보기

- 문법: `ls [옵션] [디렉터리 디렉터리2]`
- 디렉터리의 내용을 나열하고 파일과 하위 디렉터리에 대한 정보 출력

옵션

- `-a` : 숨김 파일을 포함하여 모든 파일 목록을 출력
- `-d` : 지정한 디렉터리 자체 정보를 출력
- `-i` : inode 번호 출력
- `-l` : 상세정보 출력
- `-F` : 파일의 종류를 표시
- `-h` : 파일크기값을 가독성있게 출력
- `-R` : 하위 디렉터리 목록까지 출력

ex. `ls /etc /tmp`

`ls -l /etc`

`ls -ld /etc`

`ls -al /etc`

`ls -lR /etc`

3.3 디렉터리 내용 보기

상세 정보 출력

```
[student@student ~]$ ls -l
```

```
합계 36
```

```
drwxr-xr-x. 2 student student 4096 2월 13 22:14 공개
drwxr-xr-x. 2 student student 4096 2월 13 22:14 다운로드
drwxr-xr-x. 2 student student 4096 2월 13 22:14 문서
drwxr-xr-x. 2 student student 4096 2월 13 22:14 바탕화면
drwxr-xr-x. 2 student student 4096 2월 13 22:14 비디오
...
```

문자	파일 종류
-	일반 파일
d	디렉터리
l	심벌릭 링크
b	블록 장치 파일
c	문자 장치 파일
p	파이프 파일
s	소켓 파일

필드번호	필드 값	의미
1	d	파일 종류
2	rw-r-xr-x	접근 권한
3	2	하드링크 개수
4	student	파일 소유자
5	student	파일이 속한 그룹
6	4096	파일 크기(바이트)
7	2월 13 22:14	마지막 수정 시간
8	공개	파일 이름

3.4 파일 내용 보기

파일 내용 보기

- 문법: `cat -n 파일명 파일명2 ..`
- `-n` 옵션은 행 번호를 추가해서 출력

ex. `cat /var/log/auth/log`
`cat /etc/passwd /etc/group`
`cat -n /etc/passwd`
`cat /etc/host > my_host.txt`

3.4 파일 내용 보기

화면 단위 파일 내용 보기

- 문법: `less` 파일명
- 파일 내용을 화면 단위로 출력
- 명령어
 - 화살표 키 : 한 줄씩 이용
 - `Ctrl + F` : 다음 페이지 이동
 - `Ctrl + B` : 이전 페이지 이동
 - `g` : 문서 처음으로 이동
 - `G` : 문서 끝으로 이동

ex. `less /etc/services`

3.4 파일 내용 보기

파일 뒷부분 내용 보기

- 문법: `tail [옵션] 파일명`
- 파일의 뒤 부분 몇 행을 출력
- 옵션
 - `+행번호` : 지정된 행번호부터 끝까지 출력
 - `-n 갯수` : 화면에 출력할 행의 개수 (기본값은 10)
 - `-F` : 지정된 파일변경사항을 실시간으로 확인 가능

ex. `tail /etc/passwd`
`tail -n 4 /etc/passwd`
`tail +2 /etc/passwd`

3.4 파일 내용 보기

파일 앞부분 내용 보기

- 문법: head [옵션] 파일명
- 파일의 앞 부분 몇 행을 출력
- 옵션
 - -n 갯수 : 화면에 출력할 행의 개수 (기본값은 10)

ex. head /etc/passwd
head -n 4 /etc/passwd

3.4 파일 내용 보기

주기적으로 반복 실행되는 리눅스 명령어 보기

- 문법: `watch [옵션] 파일명`
- 주기적으로 반복 실행되는 리눅스 명령어 확인
- 옵션
 - `-n 초` : 실행주기 (초)
 - `-d` : highlight 기능

ex. `watch ls`
`watch -n 1 -d ifconfig`

3.5 파일 및 디렉터리 생성

파일 생성

- 문법: touch 파일명
- 없으며 새로 생성하고 있으면 생성시간 변경됨

ex. touch report.txt
touch logs

3.5 파일 및 디렉터리 생성

디렉터리 생성

- 문법: `mkdir [옵션] 디렉터리명 디렉터리명2 ..`
- `-p` 옵션은 계층적 디렉터리 생성 가능

ex. `mkdir main sub`
`mkdir -p first/second/third`

3.5 파일 및 디렉터리 생성

파일과 디렉터리 이름 규칙

- 파일과 디렉터리 이름에는 /를 사용할 수 없다. /는 경로명에서 구분자로 사용하기 때문이다.
- 파일과 디렉터리 이름에는 알파벳, 숫자, 하이픈(-), 밑줄(_), 점(.)만 사용한다.
- 파일과 디렉터리 이름에는 공백 문자, *, |, ", ' ; @, #, \$, %, ^, & 등을 사용하면 안 된다.
- 파일과 디렉터리 이름의 영문자는 **대문자와 소문자를 구별**하여 다른 글자로 취급한다.
- 파일과 디렉터리 이름이 **.'으로 시작하면 숨김 파일**로 간주한다.

3.6 파일 및 디렉터리 복사

파일 및 디렉터리 복사

- 문법: `cp [옵션] 원본 대상`
- 원본 및 타겟은 파일 또는 디렉터리가 될 수 있음
- `-i` : interactive 기능
- `-p` : 원본파일의 속성까지 그대로 유지되어 복사됨 ex. 수정시간/접근시간/권한/소유자 등
- `-r` : 원본중에 디렉터리가 있는 경우

ex. `cp /etc/passwd ./users.txt`
`cp hello.txt users.txt first/`
`cp -r main hello.txt first/`

3.7 파일 및 디렉터리 이동 및 이름변경

파일 및 디렉터리 이동

- 문법: `mv [옵션] 원본 대상`
- 이동과 이름변경 모두 `mv` 명령어를 사용
- `-i` : interactive 기능
- `-n` : no overwrite 기능으로 기존 파일이 있으면 덮어쓰지 않음
- `-u` : 원본이 대상보다 더 최신일 때만 처리됨

ex. `mv a.txt backup/`
`mv a.txt new_b.txt`
`mv a.txt b.txt backup/`

3.8 파일 및 디렉터리 제거

파일 및 디렉터리 제거

- 문법: `rm [옵션] 경로/파일 경로/파일 ..`
- 휴지통 기능이 없기 때문에 삭제 시 주의 필요
- `-i` : interactive 기능
- `-rf` : 디렉터리 내부까지 재귀적으로 강제 삭제

```
ex.  rm dir1/abc.txt  dir1/dir2/a.txt
      rm -i  dir1/b.txt
      rm -rf dir1/
      rm -ri file1 dir1/
```


3.9 파일 및 디렉터리 찾기

파일 및 디렉터리 찾기

- 문법: `find` 경로 [옵션] 파일
- `-name` : 이름검색 (대소문자 구분)
- `-iname` : 이름검색 (대소문자 무시)
- `-type f` : 파일
- `-type d` : 디렉터리
- `-size` 크기 : 파일크기 기준
- `-user` 사용자 : 사용자 기준
- `-exec` 명령어 : 명령어 실행
- `-delete` : 삭제
- `-ls` : `find`후에 `ls` 실행
- `-maxdepth n` : 깊이 제한

ex. `find . -name todo.txt`
`find . -name todo.txt -delete`
`find /etc -name passwd`
`find /etc -type f`

3.10 파일 내용 검색

파일 내용 검색하기

- 문법: `grep` [옵션] 패턴 [파일명]
- 텍스트 검색의 핵심 명령어 (로그분석, 에러추적, 데이터 필터링에 필수)

- `-n` : 줄번호
- `-c` : 개수
- `-r` : 재귀 검색
- `-v` : 반대 검색 (inverse)
- `-E` : 확장정규식
- `-I` : 대소문자 무시

ex. `grep "ERROR" app.log`
`grep -n ERROR app.log`
`grep -c ERROR app.log`
`grep -v ERROR app.log`
`grep -E "ERROR|WARN" app.log`

3.11 하드 링크(Hard Links)

inode

- 파일의 실제정보를 저장하는 구조
- 구성요소
 - 파일크기
 - 권한(rwx)
 - 소유자
 - 생성/수정 시간
 - 데이터 위치 (디스크 블록)
- 확인: `ls -i`

3.11 하드 링크(Hard Links)

하드 링크

- 같은 inode를 공유하는 또 다른 파일 의미
- 문법: ln 원본파일 하드링크파일
 - ex. ln a.txt b.txt
- 목적:
 - 같은 파일을 '복사 없이' 여러 이름으로 공유하기 위함
 - 백업 기능 (빠르고 안전)
 - 파일 보호 (삭제 방지 효과, 한쪽 삭제해도 유지됨)

3.12 소프트 링크(Soft Links)

소프트 링크

- 원본 파일을 가리키는 바로가기(포인터) 역할
- 경로(path)를 저장하고 inode 공유 안됨
- 문법: `ln -s 원본파일 소프트링크파일`
 - ex. `ln -s a.txt b.txt`
- 목적:
 - 경로 단축
 - 여러 환경에서 동일한 설정파일 관리
 - 배포 시스템
 - ex. `ln -s app_v2 current`

3.13 파일 묶음 및 압축

tar

- 파일들을 하나로 묶음
- 문법: `tar -cvf 묶음.tar 파일1 파일2 ..`
ex. `tar -cvf test.tar a.txt b.txt`

gzip

- 파일을 압축
- 문법: `gzip 파일명 (파일명.gz)`
ex. `gzip test.tar`

tar + gzip

- 파일을 묶고 압축
- 문법: `tar -czvf test.tar.gz 폴더명`
`tar -xzvf test.tar.gz [-C 폴더명]`

ex. `tar -czvf test.tar.gz testdir/`
`tar -xzvf test.tar.gz`

4.1 파이프와 리다이렉션

파이프(Pipes)

- 문법: 명령어1 | 명령어2
- 명령어1의 출력(stdout)을 명령어2의 입력(stdin)으로 전달

ex. `ps -ef | grep java`
`ls -lhs /etc | head -n 20`
`cat -n /var/log/auth.log | grep "authentication failure"`
`cat -n /var/log/auth.log | grep "authentication failure" | wc -l`

4.1 파이프와 리다이렉션

리다이렉션

- 문법: 명령어 > 파일 (>, >>, 2>, <)
- 명령어의 입출력 방향을 바꾸는 방법

ex.

```
ls > file.txt
```

```
ifconfig >> ifconfig.txt
```

```
ifconfig | grep ether > MAC.txt
```

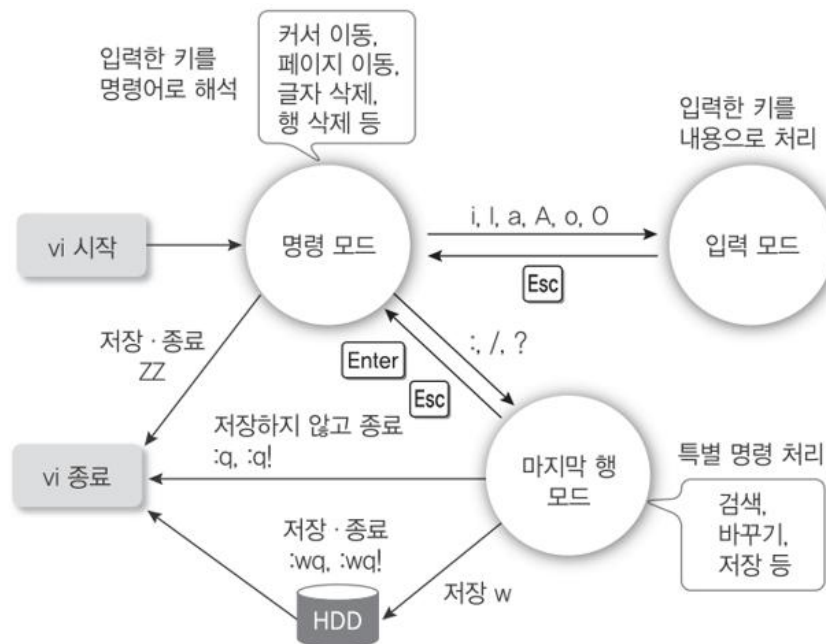
```
wc -l < file.txt
```

```
ls text.txt wrong.txt 2> error.txt
```


5.1 vi 편집기

vi 동작 모드

- 입력 모드와 명령 모드로 구분
- 입력 모드는 텍스트를 입력할 수 있는 모드이고 명령 모드는 텍스트를 삭제하고 복사/붙이기 등 편집을 하는 모드



명령 키	기능
i	커서 앞에 입력한다(현재 커서 자리에 입력한다).
a	커서 뒤에 입력한다(현재 커서 다음 자리에 입력한다).
o	커서가 위치한 행의 다음 행에 입력한다.
I	커서가 위치한 행의 첫 칼럼으로 이동하여 입력한다.
A	커서가 위치한 행의 마지막 칼럼으로 이동하여 입력한다.
O	커서가 위치한 행의 앞 행에 입력한다.

5.1 vi 편집기

주요 명령어

- 빠른 이동
 - gg : 맨위
 - G : 맨뒤
- 삭제/수정
 - dd: 한 줄 삭제
 - 2dd: 2줄 삭제
 - yy: 한 줄 복사
 - p : 붙여넣기
 - u: 작업취소
- 검색어 찾기
- 셸 실행
 - :sh

6.1 사용자 관리

사용자

- 리눅스는 다중 사용자 시스템(multi-user system)
- 권한이 없는 사용자 (non-privileged users)
 - 사용자 홈 디렉터리에서만 작업이 가능
- root 사용자 (superuser 또는 administrator)
 - 어떤 명령도 실행이 가능
 - 어떤 파일도 접근이 가능
 - 사실 Linux 시스템에서 모든 것을 할 수 있음
- 계정정보 파일
 - /etc/passwd
 - 계정이름:암호화된암호(x:다른파일에 저장의미):계정ID(UID):그룹ID(GID_:comment:home디렉터리:shell
- 비밀번호 파일
 - /etc/shadow

6.1 사용자 관리

그룹

- 사용자들을 묶어서 관리하는 방법
- 그룹에 권한을 설정하면, 그 그룹에 속한 여러 사용자 관리가 가능해짐
 - 개발팀: dev 그룹, 운영팀: ops 그룹
- 그룹 종류
 - Primary Group
 - 사용자의 기본 그룹
 - 명령어: id [사용자]
 - Secondary Group
 - 추가로 속한 그룹
 - 명령어: groups [사용자]
- 그룹 정보
 - /etc/group 에서 확인 가능

6.1 사용자 관리

사용자 생성 및 삭제/비번설정

- 문법: `sudo useradd [옵션] 사용자명`
`sudo passwd 사용자명`
`sudo userdel -r 사용자명`
- 옵션없이 지정하면 홈 디렉터리가 생성 안되고 `/bin/sh` 설정 및 사용자명이 기본 그룹으로 생성.
- `-m` : 홈 디렉터리 생성 (`/home/사용자`)
- `-d` : 명시적으로 홈 디렉터리 경로 지정
- `-s` : 셸 명시
- `-g` : Primary Group 지정
- `-G` : Secondary Group 지정

ex . `sudo useradd -m user2`
`sudo useradd -m -d /home/project/user2 user2`
`sudo useradd -m -g dev user2`
`sudo useradd -m -G sudo,admin user2`
`sudo useradd -m -s /bin/bash user2`

7.1 파일 접근 권한 관리

파일 접근 권한 보호

- 리눅스는 파일에 무단으로 접근하는 것을 방지하고 보호하는 기능을 제공함.
- 사용자는 자신의 파일과 디렉터리 중에서 다른 사용자 접근해도 되는 것과 안되는 것을 구분하여 접근 권한을 제한함

```
user1@ubuntu:~$ ls -l /etc/hosts
-rw-r--r-- 1 root root 158 8월 6 2012 /etc/hosts
user1@ubuntu:~$
```

번호	속성 값	의미
1	-	파일의 종류(- : 일반 파일, d : 디렉터리)
2	rw-r--r--	파일을 읽고 쓰고 실행할 수 있는 접근 권한 표시
3	1	하드 링크의 개수
4	root	파일 소유자의 로그인 ID
5	root	파일 소유자의 그룹 이름
6	158	파일의 크기(바이트 단위)
7	8월 6 2012	파일이 마지막으로 수정된 날짜
8	/etc/hosts	파일명 ○

7.1 파일 접근 권한 관리

접근 권한의 종류

- 읽기 권한, 쓰기 권한, 실행 권한 등 3가지로 구성

권한	파일	디렉터리
읽기	파일을 읽거나 복사할 수 있다.	ls 명령으로 디렉터리 목록을 볼 수 있다(ls 명령의 옵션은 실행 권한이 있어야 사용할 수 있다).
쓰기	파일을 수정, 이동, 삭제할 수 있다(디렉터리에 쓰기 권한이 있어야 한다).	파일을 생성하거나 삭제할 수 있다.
실행	파일을 실행할 수 있다(셸 스크립트나 실행 파일의 경우).	cd 명령을 사용할 수 있다. 파일을 디렉터리로 이동하거나 복사할 수 있다.

- 접근 권한의 표기법

- 사용자 카테고리별로 누가 파일을 읽고 쓰고 실행할 수 있는지를 문자로 표현
- 읽기 권한은 r, 쓰기 권한은 w, 실행 권한은 x로 나타내고, 해당 권한이 없으면 -로 표기
- 사용자 카테고리별로 3가지 권한의 부여 여부를 rwx 세 문자를 묶어서 표기

```

user1@ubuntu:~$ ls -l /etc/hosts
-rw-r--r-- 1 root root 158 8월 6 2012 /etc/hosts
user1@ubuntu:~$

```

rw-

r--

r--

소유자

그룹

기타 사용자

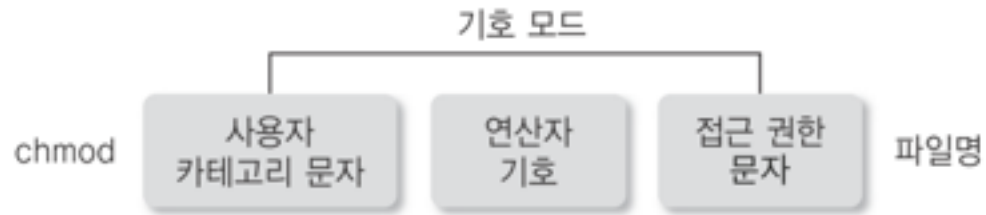
7.1 파일 접근 권한 관리

접근 권한 표기 방법 예

접근 권한	의미
<code>rw-r-xr-x</code>	소유자는 읽기, 쓰기, 실행 권한을 모두 가지고 있고 그룹과 기타 사용자는 읽기와 실행 권한만 가지고 있다.
<code>r-xr-xr-x</code>	소유자, 그룹, 기타 사용자 모두 읽기와 실행 권한만 가지고 있다.
<code>rw-----</code>	소유자만 읽기, 쓰기 권한을 가지고 있고 그룹과 기타 사용자는 아무 권한이 없다.
<code>rw-rw-rw-</code>	소유자, 그룹, 기타 사용자 모두 읽기와 쓰기 권한을 가지고 있다.
<code>rw-rwxrwx</code>	소유자, 그룹, 기타 사용자 모두 읽기, 쓰기, 실행 권한을 가지고 있다.
<code>rwx-----</code>	소유자만 읽기, 쓰기, 실행 권한을 가지고 있고 그룹과 기타 사용자는 아무 권한이 없다.
<code>r-----</code>	소유자만 읽기 권한을 가지고 있다.

7.1 파일 접근 권한 관리

파일 접근 권한 변경1- 기호모드 이용



구분	문자/기호	의미
사용자 카테고리 문자	u	파일 소유자
	g	소유자가 속한 그룹
	o	소유자와 그룹 이외의 기타 사용자
	a	전체 사용자
연산자 기호	+	권한 부여
	-	권한 제거
	=	접근 권한 설정
접근 권한 문자	r	읽기 권한
	w	쓰기 권한
	x	실행 권한

7.1 파일 접근 권한 관리

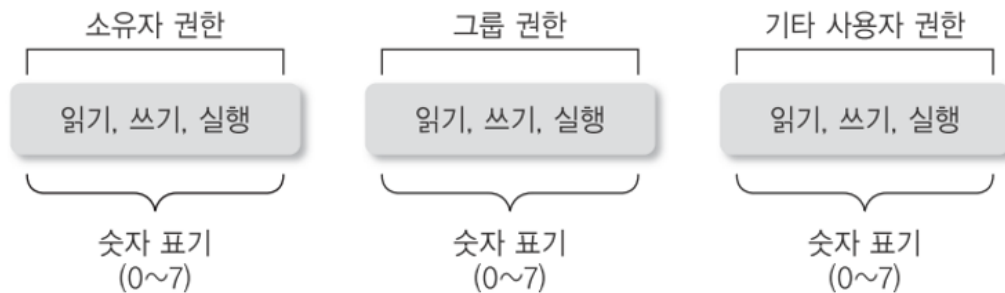
기호모드 이용 예

권한 표기	의미
u+w	소유자(u)에게 쓰기(w) 권한 부여(+)
u-x	소유자(u)의 실행(x) 권한 제거(-)
g+w	그룹(g)에 쓰기(w) 권한 부여(+)
o-r	기타 사용자(o)의 읽기(r) 권한 제거(-)
g+wx	그룹(g)에 쓰기(w)와 실행(x) 권한 부여(+)
+wx	모든 사용자에게 쓰기(w)와 실행(x) 권한 부여(+)
a+rwx	모든 사용자에게 읽기(r), 쓰기(w), 실행(x) 권한 부여(+)
u-rwx	소유자(u)에게 읽기(r), 쓰기(w), 실행(x) 권한 부여(-)
go+w	그룹(g)과 기타 사용자(o)에게 쓰기(w) 권한 부여(+)
u+x,go+w	소유자(u)에게 실행(x) 권한을 부여하고(+) 그룹(g)과 기타 사용자(o)에게 쓰기(w) 권한 부여(+)

7.1 파일 접근 권한 관리

파일 접근 권한 변경2- 숫자 모드 이용

- 숫자 모드에서는 각 권한이 있고 없고를 0과 1로 표기하고 이를 다시 환산하여 숫자로 표현
- 카테고리별로 권한의, 조합에 따라 0~7로 표현



r: 읽기, 4
w: 쓰기, 2
x: 실행, 1
-: 권한없음

ex. rwx ==> 4+2+1=7
r-x ==> 4+0+1=5
r-- ==> 4+0+0=4



7.1 파일 접근 권한 관리

숫자 모드 이용 예

접근 권한	8진수 모드	접근 권한	8진수 모드
rw-rw-rw-	777	rw-r--r--	644
rw-r-xr-x	755	rwX-----	700
rw-rw-rw-	666	rw-r-----	640
r-xr-xr-x	555	r-----	400

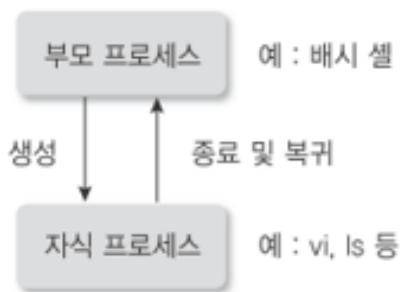
8.1 프로세스 관리

프로세스 개념

- 현재 시스템에서 실행중인 프로그램을 의미.

프로세스의 부모-자식관계

- 프로세스는 부모-자식 관계를 가짐
- 필요에 따라 부모 프로세스는 자식 프로세스를 생성하고, 자식 프로세스는 또 다른 자식 프로세스를 생성 가능
- 자식 프로세스는 할 일이 끝나면 부모 프로세스에 결과를 돌려주고 종료됨



프로세스 번호

- 각 프로세스는 고유한 번호를 가지고 있으며 이것이 PID 임.

8.1 프로세스 관리

프로세스 목록 보기

- 문법: `ps [옵션]`
- 현재 실행 중인 프로세스에 대한 정보를 출력한다.
- `-e` : 시스템에서 실행 중인 모든 프로세스의 정보를 출력.
- `-f` : 프로세스에 대한 자세한 정보를 출력.
- `-u uid` : 특정 사용자에게 대한 모든 프로세스의 정보를 출력.
- `-p pid` : pid로 지정된 특정 프로세스의 정보를 출력.
- ex. `ps -ef`

주요 명령어

- `top` # 현재 실행 중인 프로세스에 대한 정보를 주기적으로 출력.
- `fg` # 백그라운드 작업을 다시 foreground로 실행
- `bg` # 멈춰놓은 작업을 다시 background로 실행
- `ctrl + z` # 프로세스 중지
- `ctrl + c` # 프로세스 종료
- `kill pid` # 프로세스에 종료신호 보냄
- `kill -9 pid` # 강제 종료

8.2 프로세스 스케줄러

프로세스 스케줄러

- 문법: crontab [옵션] 파일명
- 정해진 시간에 특정 작업을 반복처리 지정.
- crontab 파일 형식

분(0~59)	시(0~23)	일(1~31)	월(1~12)	요일(0~6)	작업 내용
---------	---------	---------	---------	---------	-------

***** 실행명령어

```

| | | | |
| | | | └─ 요일 (0~7, 0/7=일요일)
| | | └─ 월 (1~12)
| | └─ 일 (1~31)
| └─ 시 (0~23)
└─ 분 (0~59)

```

8.2 프로세스 스케줄러

crontab 파일 형식 예

분	시	일	월	요일	의미	예시 명령
*	*	*	*	*	매 1분마다	* * * * * date
*/5	*	*	*	*	5분마다	*/5 * * * * script.sh
0	*	*	*	*	매 시간 0분	0 * * * * job.sh
0	3	*	*	*	매일 3시	0 3 * * * backup.sh
30	2	*	*	*	매일 2시 30분	30 2 * * * job.sh
0	0	*	*	0	매주 일요일 자정	0 0 * * 0 clean.sh
0	0	*	*	1	매주 월요일	0 0 * * 1 report.sh
0	12	1	*	*	매월 1일 12시	0 12 1 * * pay.sh
0	0	1	1	*	매년 1월 1일	0 0 1 1 * newyear.sh
0	9-18	*	*	1-5	평일 9~18시 매시간	0 9-18 * * 1-5 work.sh

9.1 네트워크

네트워크 필수 구성요소

- IP
- Port
- TCP/UDP
- DNS
- Gateway

필수 명령어

- `ip a` : IP 확인
- `ip route` : 기본 Gateway 확인
- `ping` : 인터넷 연결 테스트
- `nslookup` : DNS 확인
- `ss -tunlp` : port 확인
- `ifconfig` : IP 확인
- `netstat -an` : 네트워크 연결 확인
- `route -n` : Gateway 정보
- `resolvectl status`: 현재 사용중인 DNS 정보

9.1 네트워크

http에서 파일 다운로드

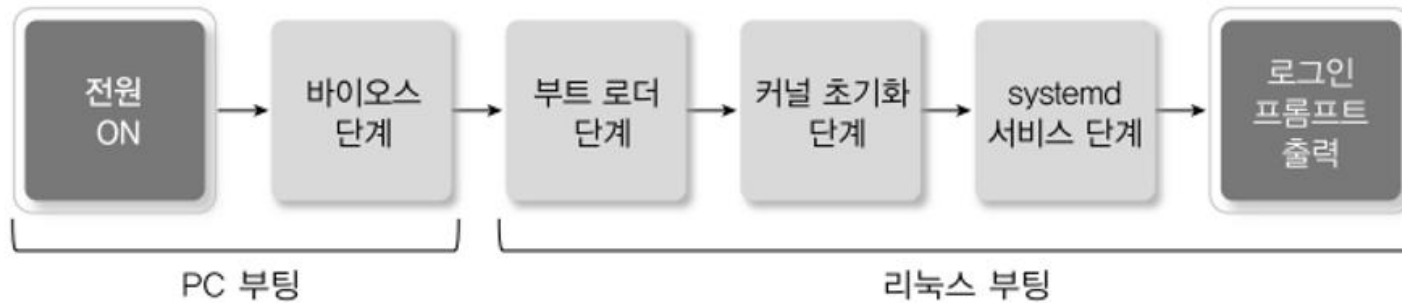
- 문법: `wget url`

샘플

- `wget -b URL` # 백그라운드로 다운로드
- `wget -c URL` # 이어 받기, -c 는 continue
- `wget -r URL` # 링크를 따라가면 재귀적으로 다운로드
- `wget -P 디렉터리 URL` # 특정 디렉터리에 저장 (대문자 P)
- `wget -O 파일명 URL` # 다른 이름으로 저장
- `wget --show-progress URL` # 진행율 확인
- `wget --limit-rate=10K URL` # 속도 제한, 초당 10k로 제한
- `wget -I down.txt` # URL 목록 파일로 지정 (여러 개 지정 가능)

10.1 시스템 부팅

리눅스 시스템 부팅 과정



1단계: 전원 ON

- 그냥 컴퓨터 켜는 순간 의미
- CPU가 실행 시작
- 아직 OS 없음

10.1 시스템 부팅

2단계: BIOS 단계

- 메인보드에 있는 프로그램 실행
- 주요 작업
 - 하드웨어 검사 (RAM, CPU, 디스크)
 - 부팅 장치 찾기 (HDD/ SDD/ USB/ CD 중 선택)
 - MBR 읽기 (Master Boot Record)
 - 부팅 프로그램(부트 로더)의 위치 저장

3단계: 부트 로더(Boot Loader)

- 설치된 여러 운영체제 중에서 부팅할 운영체제를 선택할 수 있는 메뉴 제공
- 리눅스 커널을 메모리에 로딩
- 리눅스 커널은 /boot 디렉터리에 vmlinuz-버전 형태로 제공

```
user1@ubuntu:~$ ls /boot/vm*  
/boot/vmlinuz /boot/vmlinuz-6.17.0-20-generic /boot/vmlinuz.old
```

10.1 시스템 부팅

4단계: 커널 초기화 (Kernel)

- 가장 먼저 시스템에 연결된 메모리, 디스크, 키보드, 마우스 등 장치들을 검사
- 장치 검사 등 기본적인 초기화 과정이 끝나면 프로세스와 스레드 생성.
 - 이 프로세스들은 메모리 관리 같은 커널의 여러 가지 동작을 수행.
 - 이들 프로세스는 일반적인 프로세스와 구분되도록 대괄호([])로 표시하며 PID 번호가 낮게 배정.

```
user1@ubuntu:~$ ps -ef | less
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	19:41	?	00:00:04	/sbin/init splash
root	2	0	0	19:41	?	00:00:00	[kthreadd]
root	3	2	0	19:41	?	00:00:00	[pool_workqueue_release]
root	4	2	0	19:41	?	00:00:00	[kworker/R-rcu_gp]
root	5	2	0	19:41	?	00:00:00	[kworker/R-sync_wq]
root	6	2	0	19:41	?	00:00:00	[kworker/R-kvfree_rcu_reclaim]
root	7	2	0	19:41	?	00:00:00	[kworker/R-slub_flushwq]

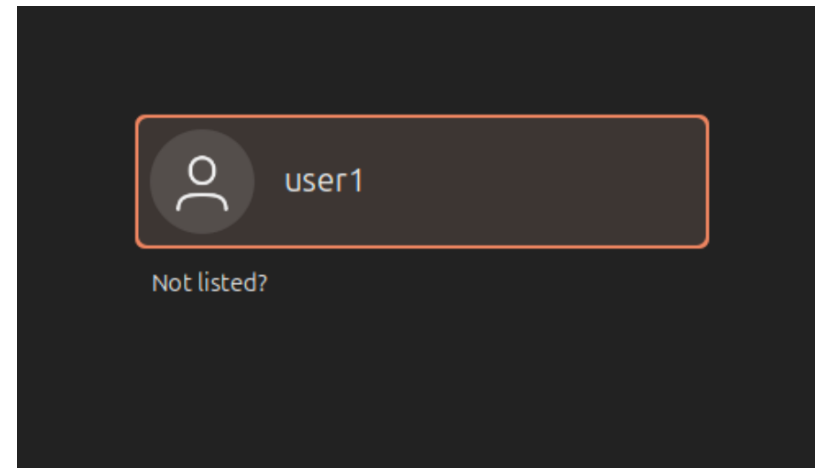
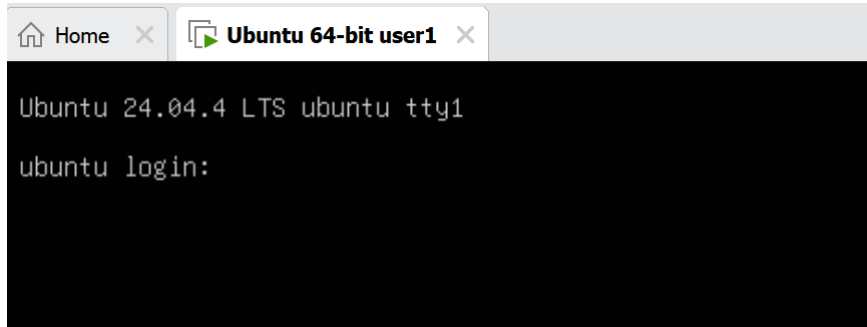
10.1 시스템 부팅

5단계: systemd 서비스 단계

- systemd 서비스 단계에 이르면 리눅스가 본격적으로 동작하기 시작.
- systemd 서비스는 init 프로세스와 동일한 기능으로서 처음 생성된 프로세스임 (PID가 1번)

6단계: 로그인 프롬프트 단계

- 마지막으로 그래픽 로그인 시스템인 GDM(GNOME Display Manager)을 동작 시키고 로그인 프롬프트 출력됨.



10.1 시스템 부팅

init 프로세스와 런레벨

- init는 시스템의 단계를 일곱 단계로 구분하고, 각 단계 따라 셸 스크립트를 실행.

런레벨	의미	관련 스크립트의 위치	런레벨	의미	관련 스크립트의 위치
0	시스템 종료	/etc/rc0.d	3	다중 사용자 모드(NFS 포함)	/etc/rc3.d
1, S	단일 사용자 모드	/etc/rc1.d	4	사용하지 않음(예비 번호)	/etc/rc4.d
2	다중 사용자 모드 (NFS를 실행하지 않음)	/etc/rc2.d	5	X11 상태로 부팅	/etc/rc5.d
			6	재시작	/etc/rc6.d

11.1 셸(shell)

셸(shell) 역할

- 사용자의 명령을 받고 운영체제의 커널이 실행되도록 하는 프로그램
- 사용자가 로그인하거나 터미널을 시작할 때 셸이 시작됨

셸(shell) 확인

- `echo $0` # 현재 사용중인 셸 정보 출력
- `cat /etc/shells` # 설치된 모든 셸 목록 출력
- `cat /etc/passwd` # 각 사용자에 대한 셸 확인

11.2 셸 스크립트

셸(shell) 스크립트

- 기본적으로 실행 가능한 텍스트 파일로서 셸 명령과 다른 특정 구조 및 구성요소를 포함함.
- ex. 변수, 함수, 루프 등

```
#!/bin/bash  
echo "Hello Linux!"
```

종류

- #!/bin/bash
- #!/bin/csh
- #!/bin/ksh
- #!/bin/sh

쉬뱅(Shebang)

- 셸 스크립트에서 #! 표현을 쉬뱅(Shebang)이라고 부름.
- /bin/bash 표현은 이 스크립트를 실행할 프로그램 경로 의미
- #!/bin/bash 는 이 파일을 bash로 실행하라는 의미

11.2 셸 스크립트

.sh 파일 실행 방법

- `$ chmod 700 first_script.sh`
- `$ ./first_script.sh`
- `$ . first_script.sh`
- `$ bash first_script.sh`
- `$ source first_script.sh`

11.3 변수 (Variables)

변수선언

- 문법: **변수명=값**
- 공백 없이 지정 필수
ex. name="홍길동"
echo "안녕하세요 \$name"

변수사용

- 문법: **\$변수명** 또는 **\${변수명}**
- 공백 없이 지정 필수
ex. name="홍길동"
echo "안녕하세요 \$name"

변수삭제

- 문법: **unset 변수명**

ex. name="홍길동"
unset name

11.3 변수 (Variables)

명령어 결과를 변수에 저장

- 문법: 변수명=\$(명령어)
- ex. today=\$(date)
- echo " \$today"

사용자 입력을 변수에 저장

- 문법: read 변수명
read -p "메시지" 변수명
- ex.
echo "이름을 입력하세요"
read name
echo "입력한 이름은 \$name"

11.3 변수 (Variables)

변수 조회

- 문법: `set | grep -w 변수명`
- ex. `set | grep -w today`

11.4 Positional Arguments

개요

- 셸 스크립트 실행 시 arguments로 전달한 값을 순서(위치)대로 저장가능.
- ex. `./hello.sh one two three`

`hello.sh : $0`

`one : $1`

`two : $2`

`three : $3`

11.5 Special Variables

개요

- 셸이 미리 만들어 둔 특별한 변수 의미
- 종류

\$0 : 파일명

\$1,\$2... : 10부터는 중괄호 지정한다. ex. \${10},\${11}

\$# : 스크립트에 전달된 인자의 갯수

\$@ : 전달된 모든 인자 전체 (개별 항목으로 유지)

\$* : 전달된 모든 인자 전체 (모든 인자를 하나의 문자열로 합침)

\$\$: 현재 실행중인 셸 또는 스크립트의 PID

\$? : 바로 직전에 실행한 명령어의 결과코드 (0:성공 , 이외값:실패)

#! : 가장 최근에 백그라운드로 실행한 프로세스 PID

11.6 조건문

개요

- 단일 if 문:

```
if [[ 조건식 ]]; then
    실행문
fi
```

- if~else 문:

```
if [[ 조건식 ]]; then
    실행문
else
    실행문
fi
```

- 다중 if 문:

```
if [[ 조건식1 ]]; then
    실행문
elif [[ 조건식2 ]]; then
    실행문
fi
```


11.6 조건문

개요

- case 문:
case 변수 in

패턴1)
실행문
;;

패턴2)
실행문
;;

*)
실행문
;;
esac

11.7 연산자

숫자 비교 연산자

연산자	의미
-eq	같다
-ne	같지 않다
-gt	크다
-lt	작다
-ge	크거나 같다.
-le	작거나 같다

ex.

```
num=10  
[[ $num -eq 10 ]]  
[[ $num -gt 5 ]]
```

11.7 연산자

문자열 비교 연산자

연산자	의미
=, ==	문자열이 같다.
!=	문자열이 같지 않다.
-z	문자열 길이가 0이다.
-n	문자열 길이가 0이 아니다.

ex.

```
name="kim"  
[ "$name" = "kim" ]  
[[ "$name" == "kim" ]]  
[[ -n "$name" ]]
```

11.7 연산자

논리 연산자

연산자	의미
&&	그리고
	또는
!	부정

ex.

```
a=8
```

```
[[ $a -gt 5 && $a -lt 10 ]]
```

```
[[ ! ($a -gt 5 && $a -lt 10) ]]
```

11.7 연산자

산술 연산자

연산자	의미
+	더하기
-	빼기
*	곱하기
/	나누기
%	나머지

ex.

```
a=10  
b=3  
echo $((a + b))  
echo $((a - b))  
echo $((a * b))  
echo $((a / b))  
echo $((a % b))
```

11.7 연산자

증감 연산자

연산자	의미
++	1증가
--	1감소

ex.

```
count=1
count=$((count + 1))
echo $count          #2
echo "> $((++count))" #3
echo "> $((--count))" #2
```

11.7 연산자

파일 검사 연산자

연산자	의미
-e	파일/디렉터리가 존재여부
-f	일반 파일 (텍스트, 로그 등)
-d	디렉터리 여부
-r	읽기 가능 여부
-w	쓰기 가능 여부
-s	크기가 0보다 크다
-L	심보릭 링크 여부

ex.

```
[ -f "test.txt" ] # 일반파일 여부  
[ -d "/tmp" ]    # 디렉터리 여부  
[ -e "hello.sh" ] # 파일 존재 여부
```

11.8 반복문

개요

- for 문:
 for 변수 in 값목록 # {1..5} | 1 2 3
 do
 실행문
 done
- for 문:
 for ((i=0; i<5; i++))
 do
 실행문
 done
- for 문:
 for ((i=0; i<5; i++))
 do
 if [[조건식]]; then
 break # continue
 fi
 done

11.8 반복문

개요

- while 문:
 while [조건식]
 do
 실행문
 done

11.9 함수

개요

- 문법:

```
function 함수명(){  
    실행문  
    [return 숫자값] # 문자열 반환은 echo 이용  
}
```

```
함수명(){  
    실행문  
}
```

호출방법: 함수명

- 파라미터:

- \$1: 첫 번째 인자
- \$2: 두 번째 인자
- \$#: 인자 개수
- \$@: 전체 인자

11.10 select 문

개요

- 사용자가 선택할 수 있는 메뉴를 자동으로 만들어주는 반복문 기능
- 문법:

```
select 변수 in 목록
do
    실행문
done
```

- 특징:
 - 자동으로 번호 메뉴 생성
 - 사용자가 번호 입력
 - 기본적으로 무한루프로 동작 (Ctrl + C 중단)
 - 선택 번호는 \$REPLY

11.11 배열

개요

- 문법:

변수=(값1 값2 값3) # 공백이 구분자, 인덱스는 0부터

```
unset 변수[0]
```

```
unset 변수
```

```
echo "${변수[0]}"
```

```
echo "${변수[@]}" # @는 전체, * 가능
```

```
echo "${변수[@]:2}" # 2부터 끝까지
```

```
echo "${변수[@]:2:3}" # 2부터 3개
```

12.1 Express 서버 배포

ubuntu에 node.js 설치하기

- <https://github.com/nodesource/distributions>

```
$ sudo su
$ apt-get update && /
apt-get install -y ca-certificates curl gnupg && /
mkdir -p /etc/apt/keyrings && /
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --dearmor -o /etc/apt/keyrings/nodesour
NODE_MAJOR=20 && /
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/node_$NODE_MAJOR.x nodistro main
/etc/apt/sources.list.d/nodesource.list && /
apt-get update && /
apt-get install nodejs -y

$ node -v
v20.20.2
```

12.1 Express 서버 배포

github로부터 Express 프로젝트 clone 하기

- https://github.com/inky4832/express_linux

```
$ git clone https://github.com/inky4832/express_linux.git  
$ cd express_linux  
$ npm install
```

.env 파일 직접 생성하기

```
$ vi .env  
DATABASE_NAME=my_database
```

12.1 Express 서버 배포

pm2 설치해서 서버 실행하고 요청하기

```
$ sudo npm i -g pm2
```

```
$ sudo pm2 start app.js
```

```
root@ubuntu:/home/user1/express_linux# sudo npm i -g pm2
added 90 packages in 6s
8 packages are looking for funding
  run 'npm fund' for details
root@ubuntu:/home/user1/express_linux# sudo pm2 start app.js

-----
      / \      / \      / \      / \      / \      / \      / \      / \
     /___\    /___\    /___\    /___\    /___\    /___\    /___\    /___\
    /     \  /     \  /     \  /     \  /     \  /     \  /     \  /
   /       \/       \/       \/       \/       \/       \/       \/       \
  /                   /                   /                   /                   \
 /                     /                     /                     /                     \
/                       /                       /                       /                       \
\                       \                       \                       \                       /
 \                     \                     \                     \                     /
  \                   \                   \                   \                   /
   \___/             \___/             \___/             \___/             \___/
    \___/             \___/             \___/             \___/             \___/
     \___/             \___/             \___/             \___/             \___/
      \___/             \___/             \___/             \___/             \___/

Runtime Edition

PM2 is a Production Process Manager for Node.js applications
with a built-in Load Balancer.

Start and Daemonize any application:
$ pm2 start app.js

Load Balance 4 instances of api.js:
$ pm2 start api.js -i 4

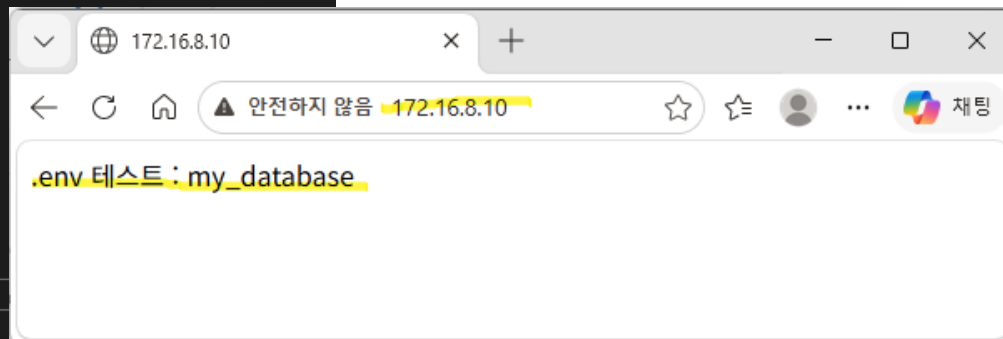
Monitor in production:
$ pm2 monitor

Make pm2 auto-boot at server restart:
$ pm2 startup

To go further checkout:
http://pm2.io/

-----
[PM2] Spawning PM2 daemon with pm2_home=/root/.pm2
[PM2] PM2 Successfully daemonized
[PM2] Starting /home/user1/express_linux/app.js in fork_mode (1 instance)
[PM2] Done.
```

id	name	namespace	version	mode	pid	uptime	⚡	status	cpu
0	app	default	1.0.0	fork	17349	0s	0	online	0%



12.2 SpringBoot 서버 배포

ubuntu에 JDK21 설치하기

```
$ sudo apt update
```

```
$ sudo apt install openjdk-21-jdk -y
```

github로부터 SpringBoot 프로젝트 clone 하기

- https://github.com/inky4832/springboot_linux

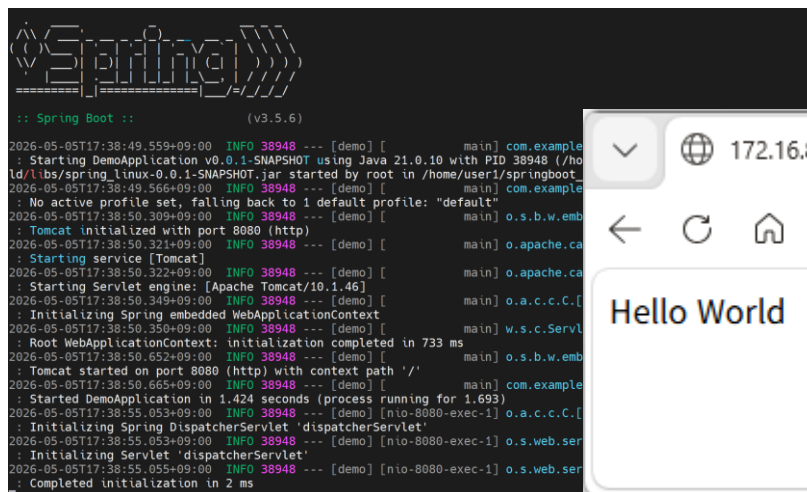
```
$ git clone https://github.com/inky4832/springboot_linux
```

```
$ cd springboot_linux
```

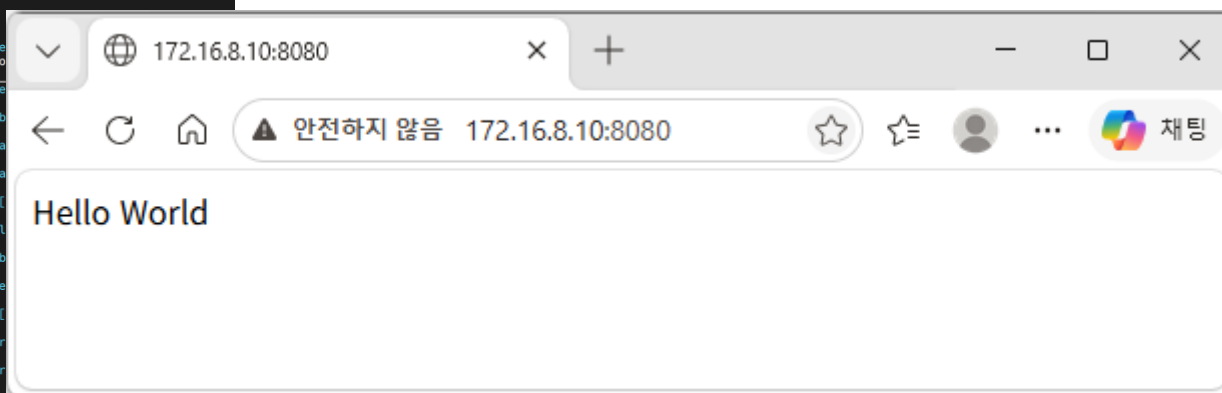

12.2 SpringBoot 서버 배포

서버 실행 하기

```
$ chmod +x gradlew  
$ ./gradlew clean build  
$ cd build/libs/  
$ sudo java -jar spring_linux-0.0.1-SNAPSHOT.jar  
# 백그라운드에서 SpringBoot 실행시키기  
$ sudo nohup java -jar spring_linux-0.0.1-SNAPSHOT.jar &
```



```
Spring Boot (v3.5.6)  
2026-05-05T17:38:49.559+09:00 INFO 38948 --- [demo] [main] com.example  
: Starting DemoApplication v0.0.1-SNAPSHOT using Java 21.0.10 with PID 38948 (/home/user1/springboot-  
2026-05-05T17:38:49.566+09:00 INFO 38948 --- [demo] [main] com.example  
: No active profile set, falling back to 1 default profile: "default"  
2026-05-05T17:38:50.309+09:00 INFO 38948 --- [demo] [main] o.s.b.w.emb  
: Tomcat initialized with port 8080 (http)  
2026-05-05T17:38:50.321+09:00 INFO 38948 --- [demo] [main] o.apache.ca  
: Starting service [Tomcat]  
2026-05-05T17:38:50.322+09:00 INFO 38948 --- [demo] [main] o.apache.ca  
: Starting Servlet engine: [Apache Tomcat/10.1.46]  
2026-05-05T17:38:50.349+09:00 INFO 38948 --- [demo] [main] o.a.c.c.C.[  
: Initializing Spring embedded WebApplicationContext  
2026-05-05T17:38:50.350+09:00 INFO 38948 --- [demo] [main] w.s.c.ServL  
: Root WebApplicationContext: initialization completed in 733 ms  
2026-05-05T17:38:50.652+09:00 INFO 38948 --- [demo] [main] o.s.b.w.emb  
: Tomcat started on port 8080 (http) with context path '/'  
2026-05-05T17:38:50.665+09:00 INFO 38948 --- [demo] [main] com.example  
: Started DemoApplication in 1.424 seconds (process running for 1.693)  
2026-05-05T17:38:55.053+09:00 INFO 38948 --- [demo] [nio-8080-exec-1] o.a.c.c.C.[  
: Initializing Spring DispatcherServlet 'dispatcherServlet'  
2026-05-05T17:38:55.053+09:00 INFO 38948 --- [demo] [nio-8080-exec-1] o.s.web.ser  
: Initializing Servlet 'dispatcherServlet'  
2026-05-05T17:38:55.055+09:00 INFO 38948 --- [demo] [nio-8080-exec-1] o.s.web.ser  
: Completed initialization in 2 ms
```

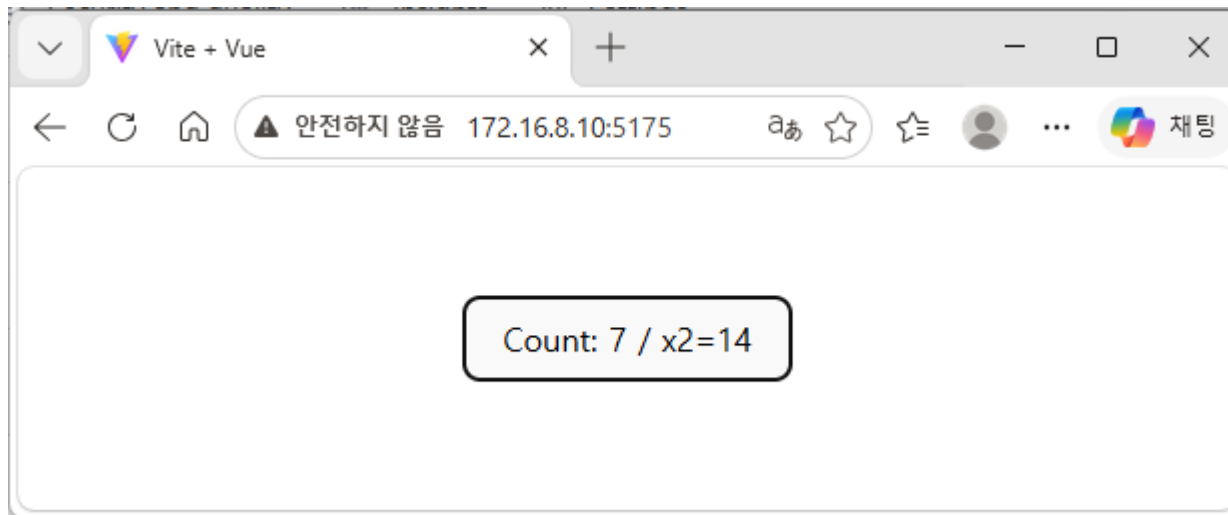


12.3 Vue.js 배포

github로부터 SpringBoot 프로젝트 clone 하기

- https://github.com/inky4832/vuejs_linux

```
$ git clone https://github.com/inky4832/vuejs_linux  
$ cd vuejs_linux/  
$ npm install  
$ npm run dev -- --host 0.0.0.0 &
```



수고하셨습니다.